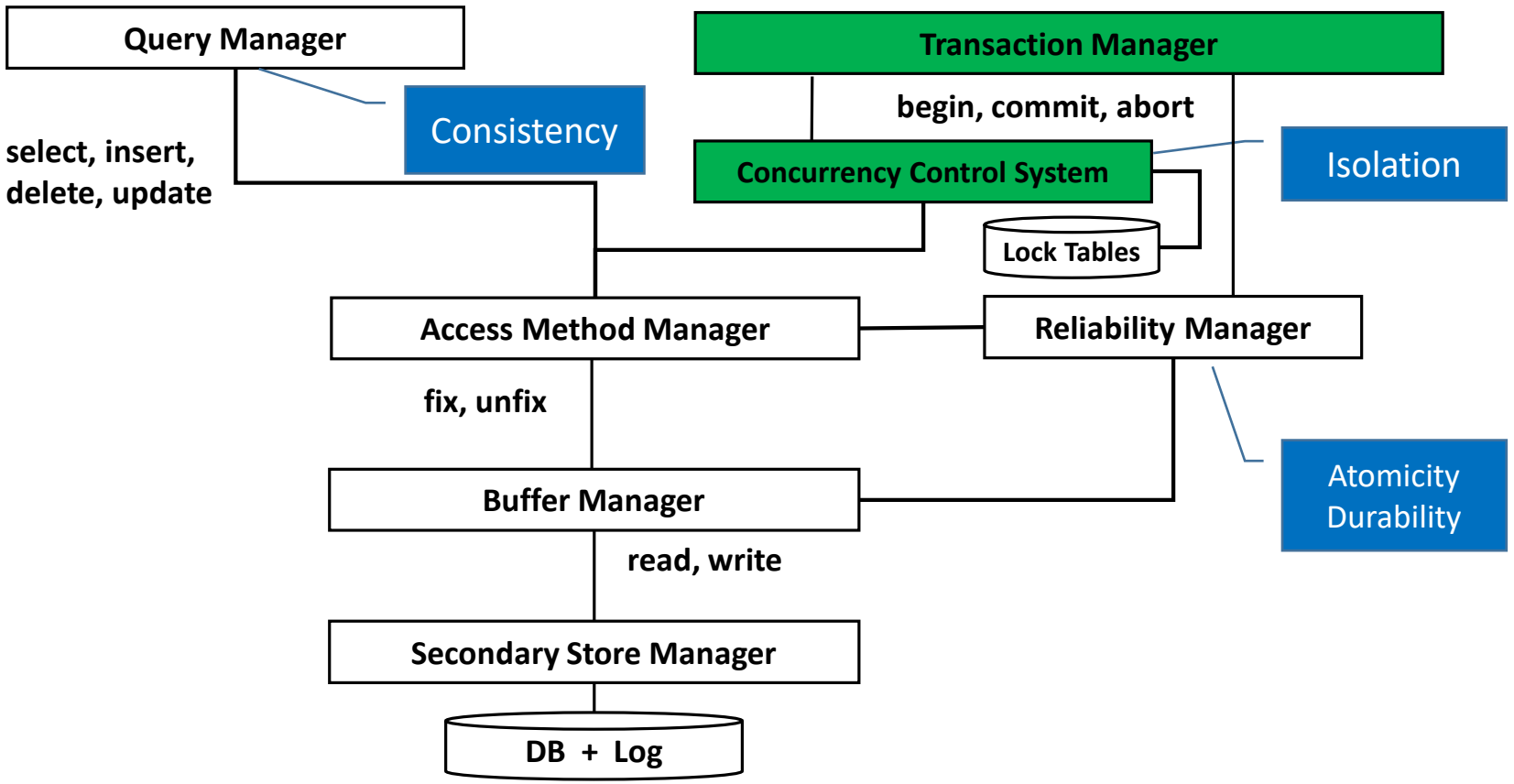
Data bases 2

Concurrency Control

Concurrency in the DBMS architecture



- The Concurrency Control System
  - Manages the simultaneous execution of transactions
  - Avoids the insurgence of anomalies
  - While ensuring performance

ONE THOUSAND TRANSACTIONS PER SECOND

Jim Gray*, Bob Good**, Dieter Gawlick***,

Pete Homan*, Harald Sammer****

Fall 1984

* Tandem, 19333 Vallco Parkway, Cupertino CA 95014

** Bank of America, Dept 3465, PO 3700,S.F. CA, 94137

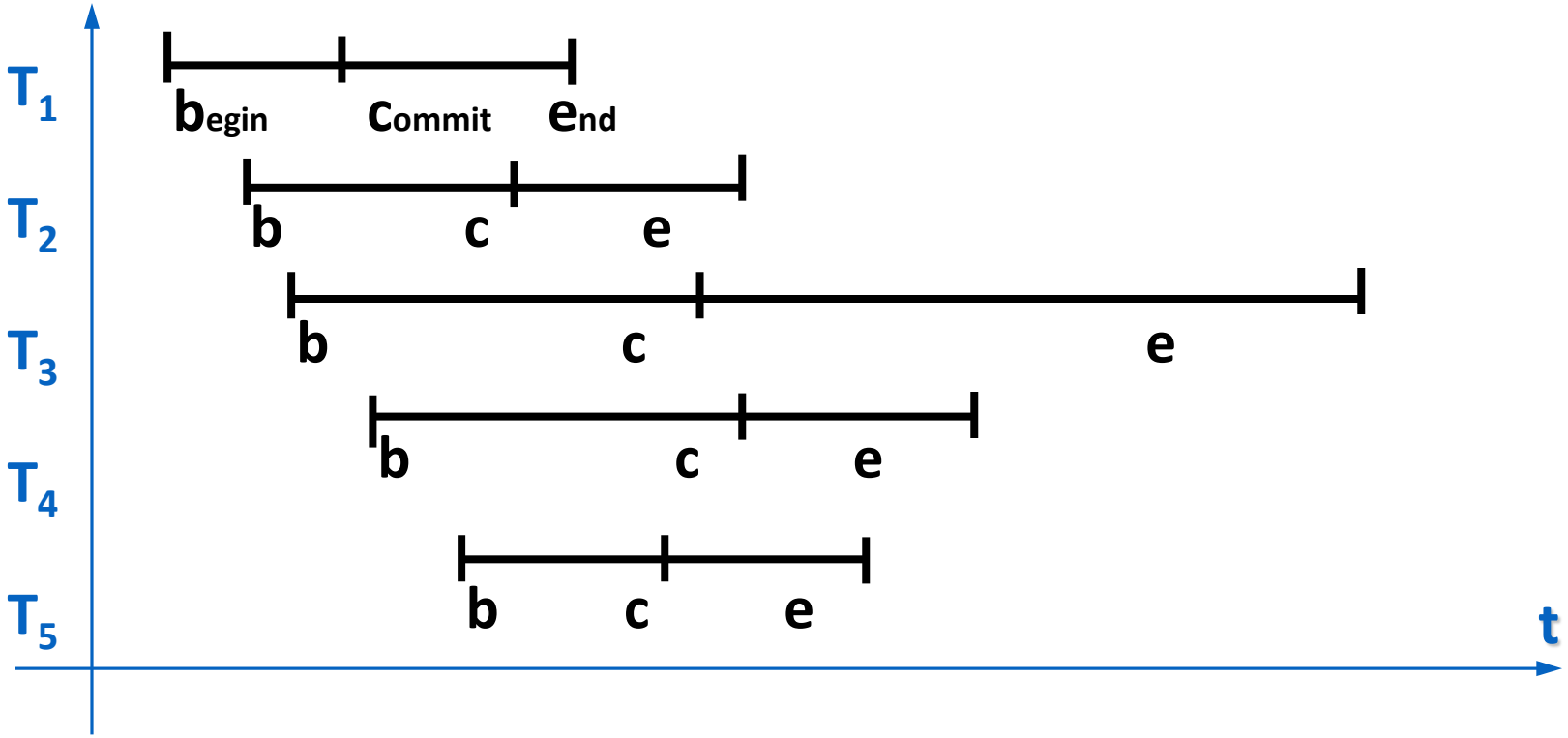
*** Amdahl, PO 3470, Sunnyvale, CA, 94088

**** Tandem, PO 560214, 6000 Frankfurt/Main 56, West Germany

ABSTRACT

Several companies intend to provide general-purpose transaction processing systems capable of one thousand transactions per second. This paper surveys the need for such systems and contrasts the approaches being taken by three different groups.

Advantages of Concurrency



- Goal: exploit parallelism to maximise transactions per second (TPS)

Problems due to Concurrency

T_1:
```
begin transaction
  update account
   set balance = balance + 3
   where customer = 'Smith'
  commit work
```

T_2:
```
begin transaction
  update account
   set balance = balance + 6
   where customer = 'Smith'
  commit work
```

Concurrent SQL transactional statements addressing the same resource

T_1:
```
begin transaction
  D = D + 3
  commit work
```

T_2:
```
begin transaction
  D = D + 6
  commit work
```

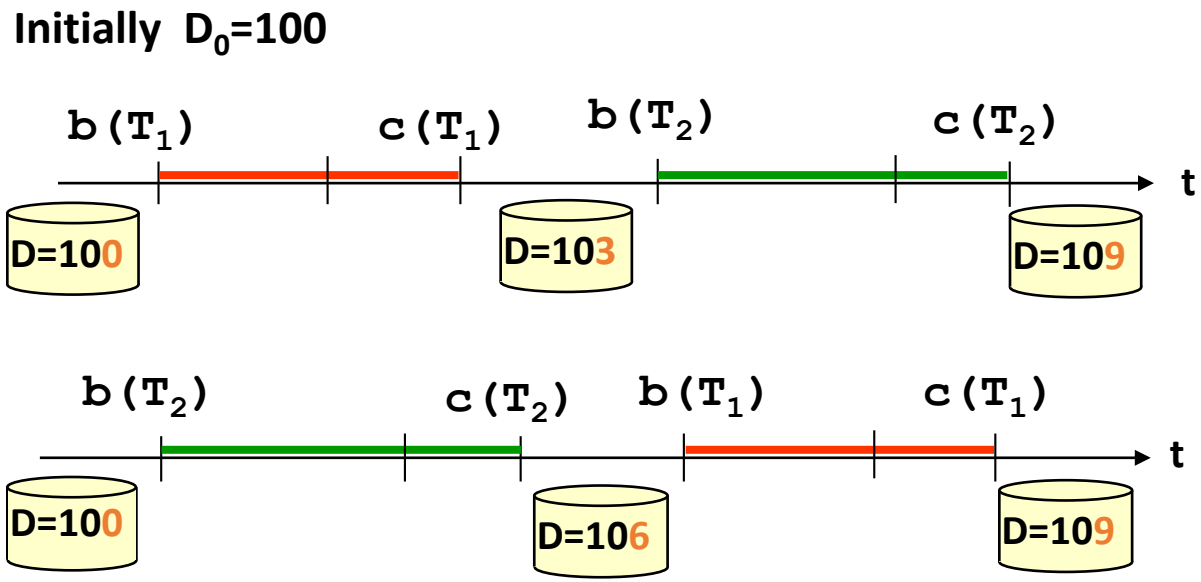
T_1:
```
begin transaction
var x;
read(D → x)
x = x + 3
write(x → D)
commit work
```

T_2:
```
begin transaction
var y;
read(D → y)
y = y + 6
write(y → D)
commit work
```

Serial Executions

```
bot(T1)
 var x;
 r(D → x)
 x = x + 3
 w(x → D)
commit(T1)
```

```
bot(T2)
 var y;
 r(D → y)
 y = y + 6
 w(y → D)
commit(T2)
```



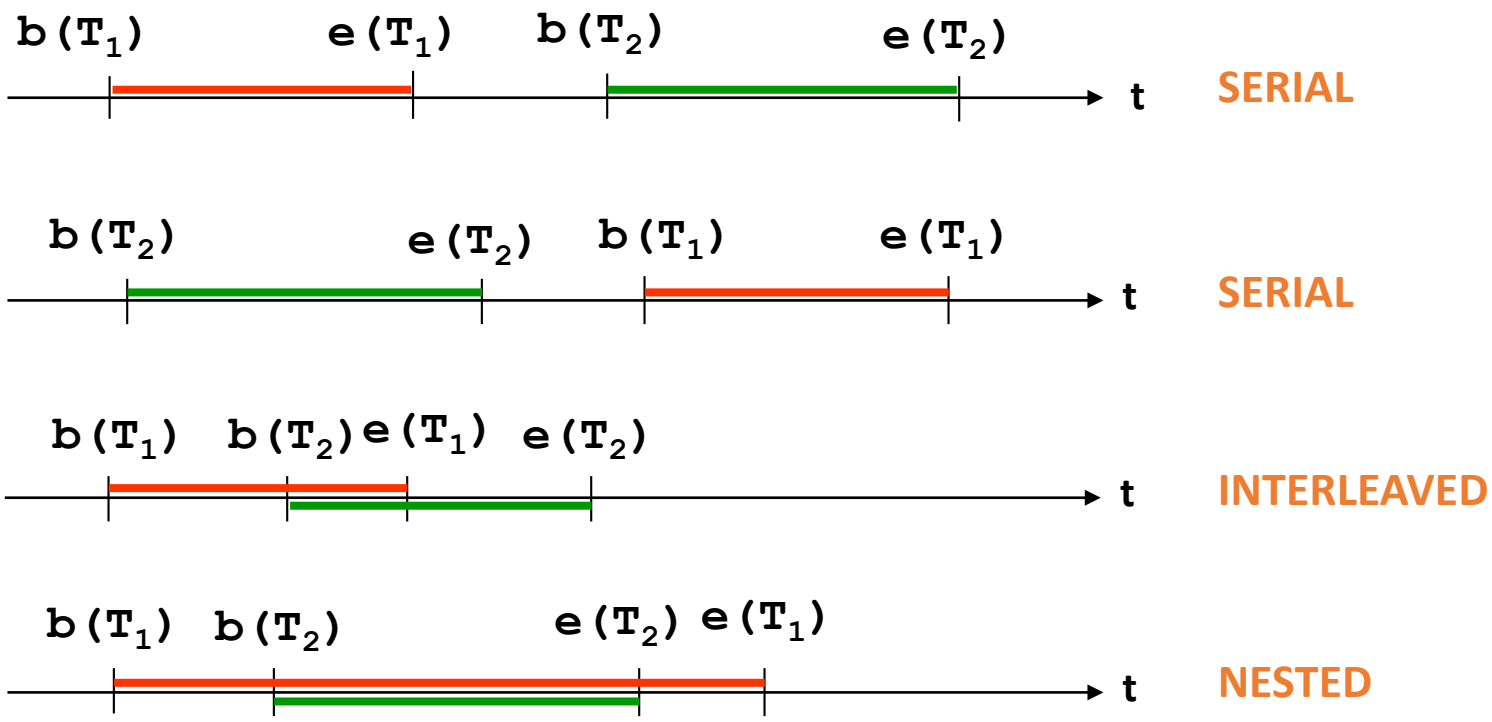
Note: it is not important that the final value be the same

Concurrency Control

- Concurrency is fundamental
  - Tens or hundreds of transactions per second cannot be executed serially
- Examples: banks, ticket reservations

- Problem: concurrent execution may cause anomalies
  - Concurrency needs to be controlled

Concurrent Executions



Execution with Lost Update

D = 100

1 T_1: r(D→ x)

2 T_1: x = x + 3

3 T_2: r(D → y)

4 T_2: y = y + 6

5 T_1: w(x → D) D = 103

6 T_2: w(y → D) D = 106 !

T1 : **UPDATE account**
SET balance = balance + 3
WHERE client = 'Smith'

T2 : **UPDATE account**
SET balance = balance + 6
WHERE client = 'Smith'

- Note: this anomaly does not depend merely on T2 overwriting the value produced by T1
  - w1(x), w2(x) is ok (serial)
  - r1(x), w1(x), w2(x) is ok too (serial)
  - r1(x), r2(x), w1(x), w2(x) is not ok: inconsistent updates **from the same initial value**

Sequence of I/O actions producing the error



or



Dirty Read

T1 : **UPDATE account**
SET balance = balance + 3
WHERE client = 'Smith'

T2 : **UPDATE account**
SET balance = balance + 6
WHERE client = 'Smith'

D = 100
1 T_1: r(D → x)
2 T_1: x = x + 3
3 T_1: w(x → D) D = 103
4 T_2: r(D → y) this read is "dirty" (uncommitted value)
5 T_1: **rollback**
6 T_2: y = y + 6
7 T_2: w(y → D) D = 109 **!**

Nonrepeatable Read

D = 100

1 T_1: r(D → x)

2 T_2: r(D → y)

3 T_2: y = y + 6

4 T_2: w(y → D) D = 106

5 T_1: r(D → **z**) z <> x !

T1 : **SELECT balance FROM account**
WHERE client = 'Smith'
…
SELECT balance FROM account
WHERE client = 'Smith'

T2 : **UPDATE account**
SET balance = balance + 6
WHERE client = 'Smith'

Phantom Update

Constraint: A+B+C=100, A=50, B=30, C=20

T_1: r(A → x), r(B → y)……
T_2: r(B → s), r(C → t)
T_2: s = s + 10, t = t – 10
T_2: w(s → B), w(t → C) (now B=40, C=10, A+B+C=100)
T_1: r(C → z) (but, for T_1, x+y+z = A+B+C = **90!**)

- So for T_1 it is as if "somebody else" had updated the value of the sum
- But for T_2 the update is perfectly legal (does not change the value of the sum)

Phantom Insert

Tab

A	B

T_1: C=AVG(B: A=1) `C = select avg(B) from Tab where A=1`

T_2: Insert (A=1,B=2) `insert into Tab values( 1, 2 )`

T_1: C=AVG(B: A=1) `C = …`

- Note: this anomaly does not depend on data already present in the DB when T1 executes, but on a "phantom" tuple that is inserted by "someone else" and satisfies the conditions of a previous query of T1

Summary of Anomalies

- Lost update r1 - r2 - w2 - w1
  - An update is applied from a state that ignores a preceding update, which is lost
- Dirty read r1 - w1 - r2 - abort1 - w2
  - An uncommitted value is used to update the data
- Nonrepeatable read r1 - r2 - w2 - r1
  - Someone else updates a previously read value
- Phantom update r1 - r2 - w2 - r1
  - Someone else updates data that contributes to a previously valid constraint
- Phantom insert r1 - w2(new data) - r1
  - Someone else inserts data that contributes to a previously read datum

Concurrency Theory vs System Implementation

- **Model**: an abstraction of a system, object or process, which purposely disregards details to simplify the investigation of relevant properties
- Concurrency Theory builds upon a model of transactions and concurrency control principles that helps understanding real systems
- Real systems exploit implementation level mechanisms (locks, snapshots) which help achieve some of the desirable properties postulated by the theory
  - Do not look for a view or conflict serializability checker in your DBMS
  - Look instead for lock tables, lock types, lock granting rules, snapshots, etc..
  - Understand how the implementation mechanisms ensure properties modelled by the Concurrency Theory

Transactions, Operations, Schedules

- **Operation**: a read or write of a specific datum by a specific transaction
  - NB: the schedule notation omits the program variables
    - r1(x) = r1(x → .)
    - w1(x) = w1(. → x)
    - r1(x) and r1(y) are different operations
    - r1(x) and r2(x) are different operations
    - w1(x) and w2(x) are different operations
- **Schedule**: a sequence of operations performed by concurrent transactions that respects the order of operations of each transaction
  - T1: r1(x) w1(x)
  - T2: r2(z) w2(z)
  - S1: r1(x) r2(z) w1(x) w2(z)

Schedules

- How many distinct schedules exist for two transactions?
  - With T1 and T2 from the previous slide:
  - r1(x) w1(x) r2(z) w2(z)
  - r2(z) w2(z) r1(x) w1(x)
  - r1(x) r2(z) w1(x) w2(z)
  - r2(z) r1(x) w2(z) w1(x)
  - r1(x) r2(z) w2(z) w1(x)
  - r2(z) r1(x) w1(x) w2(z)

$N = 6$

serial

interleaved

nested

Principles of Concurrency Control

- Goal: to reject schedules that cause anomalies
- **Scheduler**: a component that accepts or rejects the operations requested by the transactions

- **Serial schedule**: a schedule in which the actions of each transaction occur in a contiguous sequence

- S: r0(x) r0(y) w0(x) r1(y) r1(x) w1(y) r2(x) r2(y) r2(z) w2(z)



T0 T1 T2

Principles of Concurrency Control

- **In case of non-serial schedules?**

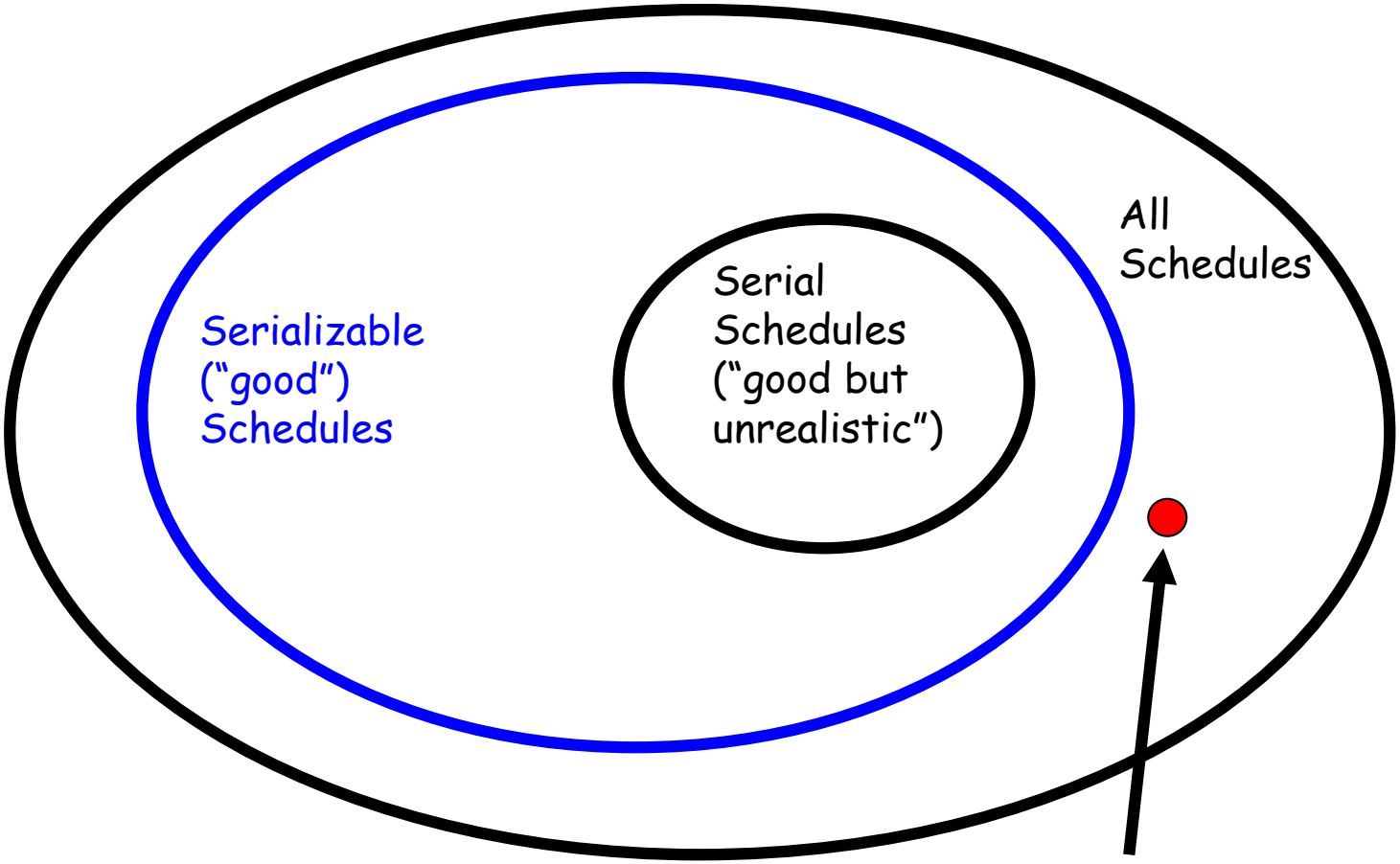
  S: r0(x) r1(y) r0(y) w0(x) r1(x) r2(x) w1(y) r2(y) r2(z) w2(z)

- **Notion of Serializable schedule**
  - A schedule that leaves the database in the same state as **some** serial schedule of the same transactions
  - This is commonly accepted as a notion of schedule **correctness**
  - Requires a notion of schedule equivalence to identify classes of schedules that ensure serializability
    - Different notions → different classes (cost of checking if schedule is serializable)

Principles of Concurrency Control

- Assumption
  - We initially assume that transactions are observed “a-posteriori” and limited to those that have committed (**commit-projection**), and we decide whether the observed schedule is admissible
  - In practice (and in contrast), schedulers must make decisions while transactions are running

Basic Idea



This schedule **may** generate anomalies.
Its execution leads to a database state that no serial execution would produce

Schedules: how many?

- n transactions (T1,T2,…,Ti,…,Tn), each with k_i operations
- T1 has k_1 operations, Ti has k_i operations…

T1: o_{11} o_{12} … $o_{1\,k1}$ … Ti: o_{i1} o_{i2} … $o_{i\,ki}$ … Tn: o_{n1} o_{n2} … $o_{n\,kn}$

- How many serial schedules (N_S) exist for n transactions?

$$N_S = n!$$

N_S = number of permutations of N transactions

- How many distinct schedules (N_D) exist for n transactions?

$N_S << N_D$

$$N_D = \frac{\left(\sum_{i=1}^{n} k_i\right)!}{\prod_{i=1}^{n}(k_i!)}$$

N_D = number of permutations of all operations / product of the number of permutations of the operations of each transaction (only 1 out of the k_i permutations is valid, the one that respects the sequence of operations)

View-serializability

- Preliminary definitions:
  - $r_i(x)$ reads-from $w_j(x)$ in a schedule S when $w_j(x)$ precedes $r_i(x)$ and there is no $w_k(x)$ in S between $w_j(x)$ and $r_i(x)$
  - $w_i(x)$ in a schedule S is a final write if it is the last write on x that occurs in S
- Two schedules are view-equivalent ($S_i \approx_V S_j$) if: they have 1) the same operations, 2) the same reads-from relationship, and 3) the same final writes
- A schedule is view-serializable if it is view-equivalent to a serial schedule of the same transactions
- The class of view-serializable schedules is named VSR
- **Mnemonically**: S is view-serializable if 1) every read operation sees the same values and 2) the final value of each object is written by the same transaction as if the transactions were executed serially in **some** order

Examples of view-serializability

- **S3**: w0(x) r2(x) r1(x) w2(x) w2(z)
- S4: w0(x) r1(x) r2(x) w2(x) w2(z)
  - r1(x) reads from w0(x); r2(x) reads from w0(x)
  - final writes on x: w2(x) on z: w2(z)





serial

- S3 is view-equivalent to serial schedule S4 (so it is view-serializable)

- **S5**: w0(x) r1(x) w1(x) r2(x) w1(z)
- S6: w0(x) r1(x) w1(x) w1(z) r2(x)
  - r1(x) reads from w0(x); r2(x) reads from w1(x)
  - final writes on x: w1(x) on z: w1(z)





serial

- S5 is view-equivalent to serial schedule S6, so it is also view-serializable

Examples of view-serializability

- S7 : r1(x) r2(x) w1(x) w2(x)
- S8 : r1(x) r2(x) w2(x) r1(x)
- S9 : r1(x) r1(y) r2(z) r2(y) w2(y) w2(z) r1(z)

- S7 corresponds to a lost update
- S8 corresponds to a non-repeatable read
- S9 corresponds to a phantom update
- They are all non view-serializable

(check by testing view equivalence with T1, T2 and with T2, T1)

A More Complex Example

S_{10} : $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ **$r_2(x)$** $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ **$w_1(x)$** $w_3(y)$

- Is S_{10} serializable? Yes iff there exists a serial schedule S_s s.t. $S_{10} \approx_V S_s$

Let's try with S_{11} : T_0 T_1 T_2 T_3

- $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_1(x)$ $r_1(z)$ **$w_1(x)$** **$r_2(x)$** $w_2(y)$ $r_3(z)$ $w_3(z)$ $w_3(y)$
  - In S_{11} **$r_2(x)$** reads-from **$w_1(x)$**
  - In S_{10} **$r_2(x)$** reads-from $w_0(x)$

Let's try with S_{12}: T_0 T_2 T_1 T_3

And also with S_{13}: T_0 T_2 T_3 T_1



A More Complex Example

S_{10}: $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

S_{12}: $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$ $r_3(z)$ $w_3(z)$ $w_3(y)$

reads-from OK: $r_1(x)$ from $w_0(x)$,
$r_1(z)$ from $w_0(z)$,
$r_2(x)$ from $w_0(x)$,
$r_3(z)$ from $w_0(z)$,

final writes OK: $w_1(x)$, $w_3(y)$, $w_3(z)$

$S_{10} \in$ VSR



A More Complex Example

S_{10}: $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

S_{13}: $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_3(z)$ $w_3(z)$ $w_3(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$

reads-from : $r_1(x)$ from $w_0(x)$,
$r_1(z)$ from $w_3(z)$

Different reads-from relationship

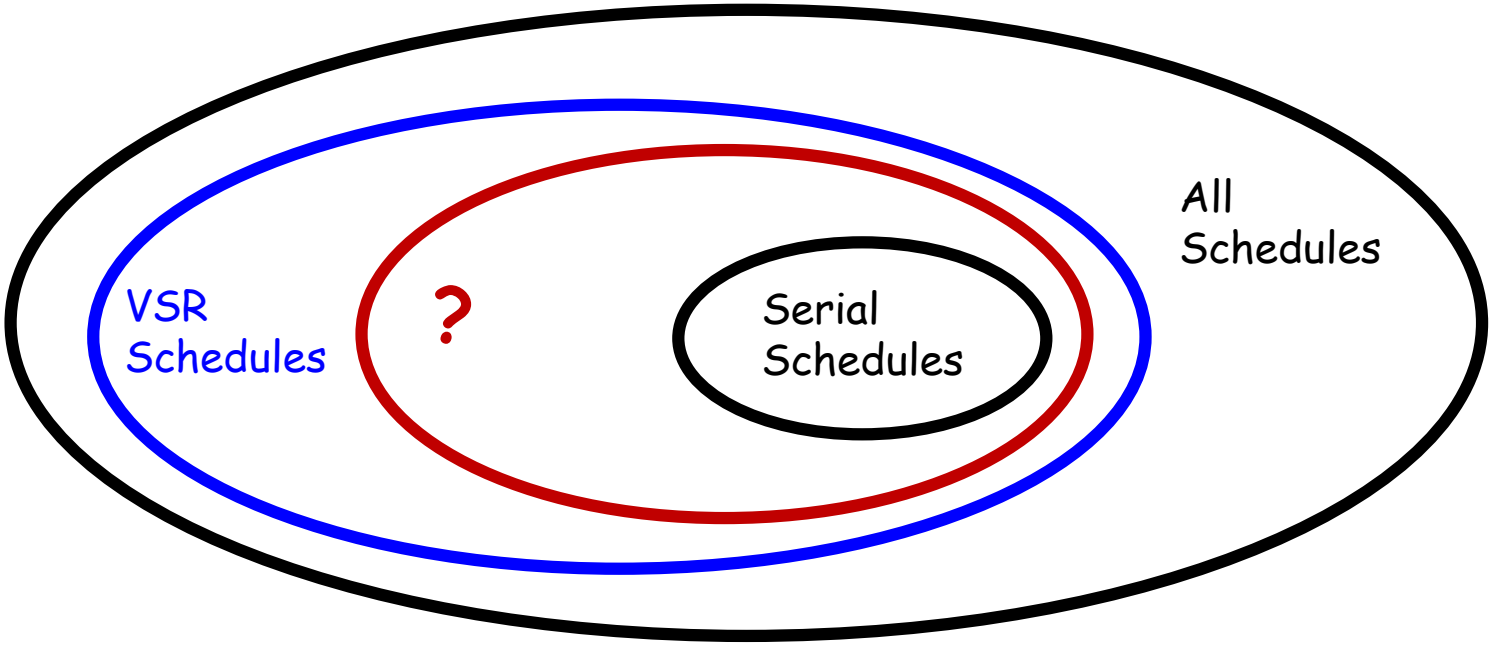
S10 not view-equivalent to T0,T2,T3,T1



Complexity of view-serializability

- Deciding view-equivalence of two given schedules is done in polynomial time and space
- Deciding if a **generic** schedule is in VSR is an **NP-complete problem**
  - requires considering the reads-from and final writes of all possible serial schedules with the same operations – combinatorial in the general case
  - Performance!! what can we trade for that?
    - …Accuracy!
- We look for a **stricter definition that is easier to check**
  - May lead to **rejecting some schedule that would be acceptable under view-serializability** but not under the stricter-faster criterion

VSR schedules are "too many"



Conflict-serializability

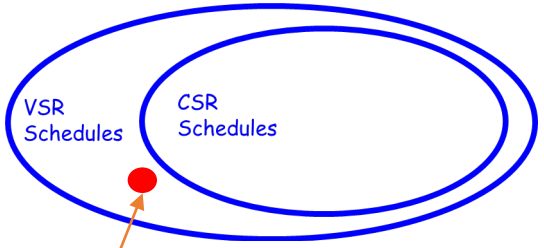
- Preliminary definition:
  - Two operations o_i and o_j ($i \neq j$) are in conflict if they address the same resource and at least one of them is a write
    - read-write conflicts (r-w or w-r)
    - write-write conflicts (w-w)

Conflict-serializability

- Two schedules are conflict-equivalent ($S_i \approx_C S_j$) if :
  - S_i and S_j contain the same operations and in all the conflicting pairs the transactions occur in the same order
- A schedule is conflict-serializable iff it is conflict-equivalent to a serial schedule of the same transactions
- The class of conflict-serializable schedules is named CSR

Relationship between CSR and VSR

- **VSR ⊃ CSR** : all conflict-serializable schedules are also view-serializable, but the converse is not necessarily true



- Proof: there are VSR schedules not in CSR
- Counter-example: we consider $r_1(x)\ w_2(x)\ w_1(x)\ w_3(x)$ that
  - is view-serializable: it is view-equivalent to

    $T_1T_2T_3 = r_1(x)\ w_1(x)\ w_2(x)\ w_3(x)$
  - is not conflict-serializable, due to the presence of

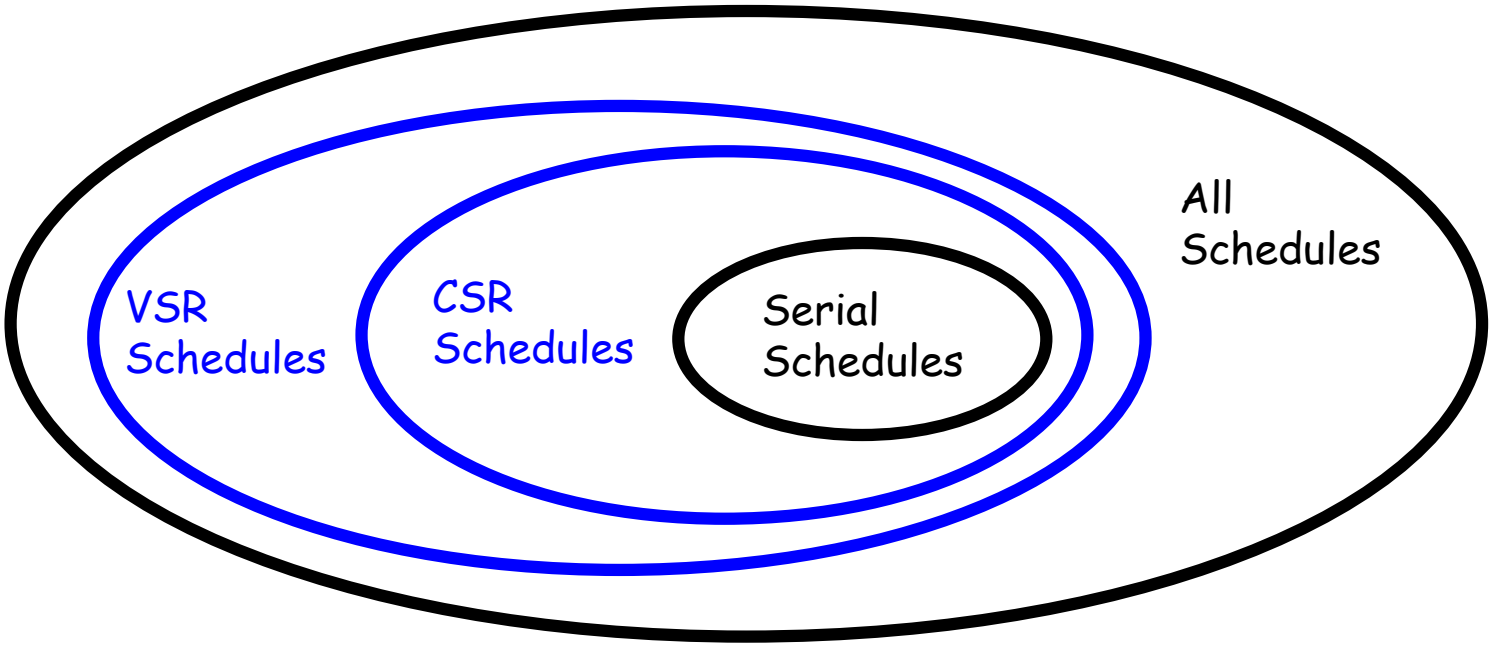
    $r_1(x)\ w_2(x)$ and $w_2(x)\ w_1(x)$

    these two operations cannot be conflict-equivalent to T_1T_2 or T_2T_1

CSR implies VSR

- CSR → VSR: conflict-equivalence $\approx_C$ implies view-equivalence $\approx_V$
- We assume S1 $\approx_C$ S2 and prove that S1 $\approx_V$ S2
- S1 and S2 must have:
  - The same final writes: if they didn't, there would be at least two writes in a different order, and since two writes are conflicting operations, the schedules would not be $\approx_C$
  - The same "reads-from" relations: if not, there would be at least one pair of conflicting operations in a different order, and therefore, again, $\approx_C$ would be violated

CSR and VSR


All
Schedules
VSR
Schedules
CSR
Schedules
Serial
Schedules

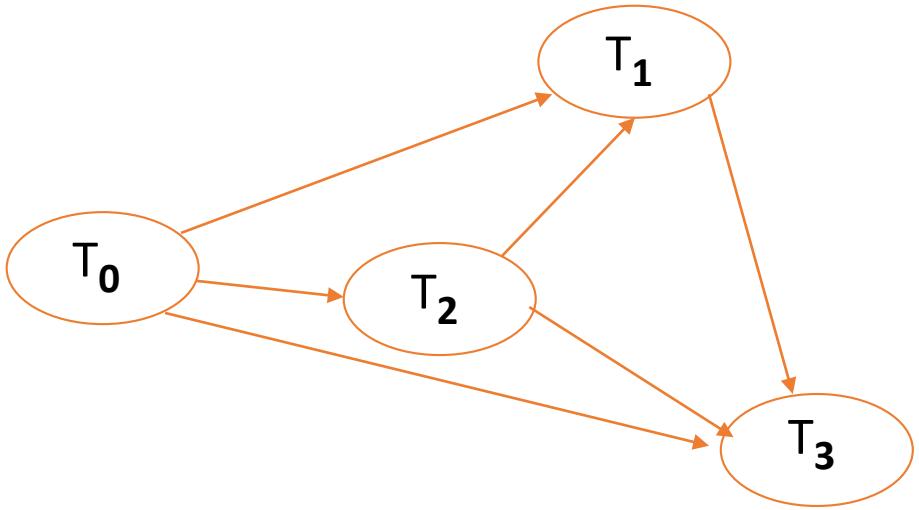
Testing conflict-serializability

- Is done with a *conflict graph* that has:
  - One node for each transaction T_i
  - One arc from T_i to T_j if there exists at least one conflict between an operation o_i of T_i and an operation o_j of T_j such that o_i precedes o_j
- Theorem:
  - A schedule is in CSR if and only if its conflict graph is acyclic

Testing conflict-serializability

S_{10} : $w_0(x)\ r_1(x)\ w_0(z)\ r_1(z)\ r_2(x)\ w_0(y)\ r_3(z)\ w_3(z)\ w_2(y)\ w_1(x)\ w_3(y)$

- Resource-based projections:
  - **x** : $w_0\ r_1\ r_2\ w_1$
  - **y** : $w_0\ w_2\ w_3$
  - **z** : $w_0\ r_1\ r_3\ w_3$

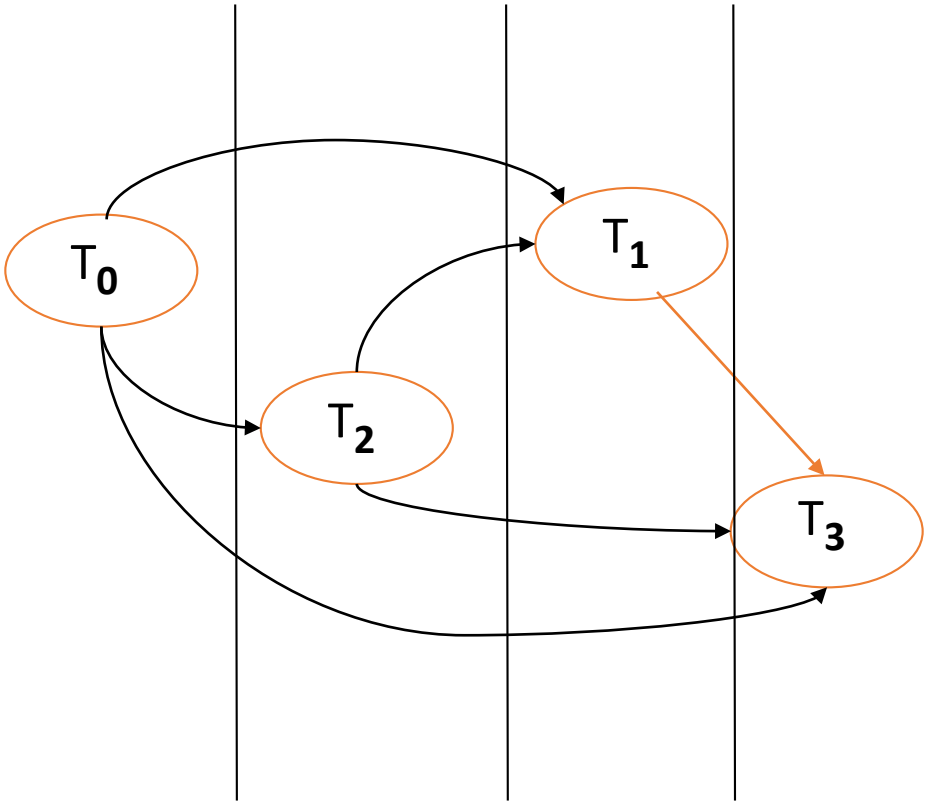


CSR implies acyclicity of the CG

- Consider a schedule S in CSR. As such, it is $\approx_C$ to a serial schedule
- W.l.o.g. we can (re)label the transactions of S to say that their order in the serial schedule is: T1 T2 … Tn
- Since the serial schedule has all conflicting pairs in the same order as schedule S, in the conflict graph there can only be arcs (i,j), with i<j
- Then the graph is acyclic, as a cycle requires at least an arc (i,j) with i>j

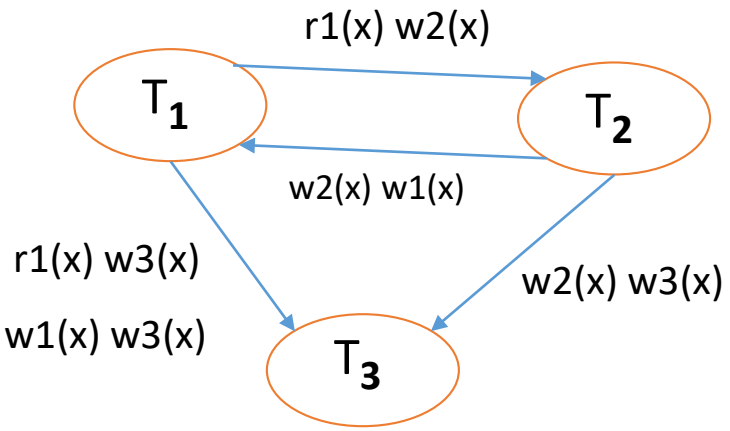
Acyclicity of the CG implies CSR

- If S's graph is acyclic then it induces a topological (partial) ordering on its nodes, i.e., an ordering such that the graph only contains arcs (i,j) with i<j. The same partial order exists on the transactions of S
- Any serial schedule whose transactions are ordered according to the partial order is conflict-equivalent to S, because for all conflicting pairs (i,j) it is always i<j
  - In the example before: T0 < T2 < T1 < T3
  - In general, there can be many compatible serial schedules (i.e., many serializations for the same acyclic graph)
  - As many as the total orders compatible with the partial topological order

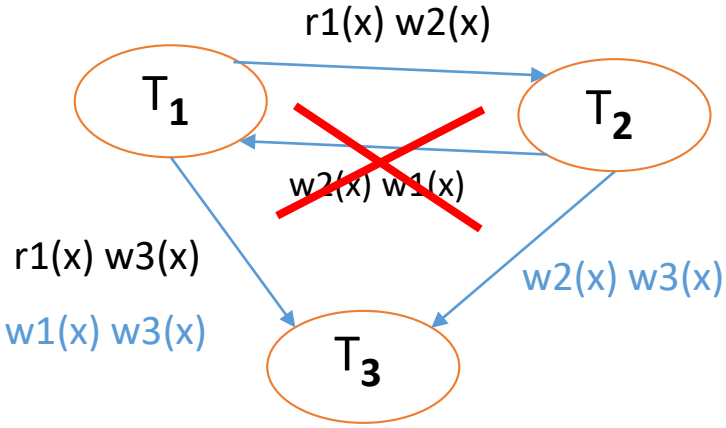


Let's go back… (2)

r1(x) w2(x) w1(x) w3(x)



RW – WR – WW conflicts



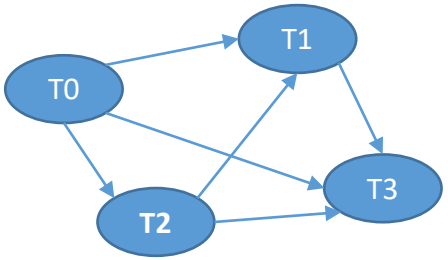
Reads-from and final write constraints

Back to: A More Complex Example

(VSR)

S_{10}: $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

- $w_0(x)$ $r_1(x)$ $r_2(x)$ $w_1(x)$ → T0<T1, T0<T2, T2<T1
- $w_0(y)$ $w_2(y)$ $w_3(y)$ → T0<T3, T2<T3
- $w_0(z)$ $r_1(z)$ $r_3(z)$ $w_3(z)$ → T0<T1, T0<T3, T1<T3

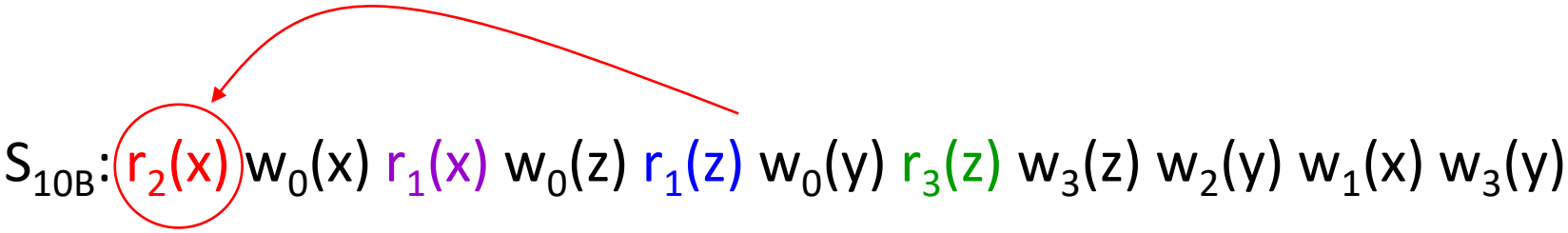


final write

→ T_0 T_2 T_1 T_3

S_{12}: $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$ $r_3(z)$ $w_3(z)$ $w_3(y)$

TO DO (at home): analyze CRS and VSR



Concurrency Control in Practice

- CSR checking would be efficient if we knew the graph from the beginning — but we don't
- A scheduler must rather work "**online**", i.e., decide for each requested operation whether to execute it immediately or to reject/delay it
- It is not feasible to maintain the conflict graph, update it, and check its acyclicity at each operation request
- The assumption that concurrency control can work only with the commit-projection of the schedule is unrealistic, **aborts do occur**
- Some simple on-line "decision criterion" is required for the scheduler, which must
  - **avoid as many anomalies as possible**
  - **have negligible overhead**

Arrival sequences vs a posteriori schedules

- So far the notation
  - $r_1(x)$ $w_2(x)$ $w_1(x)$ $w_3(x)$
- represented a “schedule”, which is an a posteriori view of the execution of concurrent transactions in the DBMS (also called “history” in some books)
- A schedule represents “what has happened”, “which operations have been executed by which transaction in which order”
- They can be further restricted by the **commit-projection hypothesis** to operations executed by **committed** transactions
- When dealing with “online” concurrency control, it is important also to consider “**arrival sequences**”, i.e., sequences of operation requests emitted in order by transactions
- With an abuse of notation, we will denote an arrival sequence in the same way as a posteriori schedule
  - $r_1(x)$ $w_2(x)$ $w_1(x)$ $w_3(x)$
- The distinction will be clear from the context

Concurrency control approaches

- How can concurrency control be implemented “online”?
- Two main families of techniques:
  - Pessimistic
    - Based on locks, i.e., resource access control
    - If a resource is taken, make the requester wait or pre-empt the holder
  - Optimistic
    - Based on timestamps and versions
    - Serve as many requests as possible, possibly using out-of-date versions of the data
- We will compare the two families after introducing their features
- Commercial systems take the best of both worlds

Locking

- It's the most common method in commercial systems
- A transaction is well-formed w.r.t. locking if
  - read operations are preceded by **r_lock** (aka **SHARED LOCK**) and followed by **unlock**
  - write operations are preceded by **w_lock** (aka **EXCLUSIVE LOCK**) and followed by **unlock**
  - Note: unlocking can be **delayed** w.r.t. to the end of the read/write operation

r_lock1(x) r1(x) unlock1(x)

w_lock1(x) w1(x) unlock1(x)

- Transactions that first read and then write an object may:
  - Acquire a w_lock already when reading
  - Acquire a r_lock first and then upgrade it into a w_lock (lock upgrade)

r1(x) w1(x)

w_lock1(x) r1(x) w1(x) …

- Possible states of an object:
  - free
  - r-locked (locked by one or more readers)
  - w-locked (locked by a writer)

r_lock1(x) r1(x) w_lock1(x) w1(x) …

Behavior of the Lock Manager (Conflict Table)

- The lock manager receives the primitives from the transactions and grants resources according to the conflict table
  - When a lock request is granted, the resource is acquired
  - When an unlock is executed, the resource becomes available

REQUEST	RESOURCE STATUS		
	FREE	R_LOCKED	W_LOCKED
`r_lock`	OK R_LOCKED	OK R_LOCKED (n++)	NO W_LOCKED
`w_lock`	OK W_LOCKED	NO R_LOCKED	NO W_LOCKED
`unlock`	*ERROR*	OK *DEPENDS* (n--)	OK FREE

n: counter of the concurrent readers, (inc|dec)remented at each (`r_`|`un`)`lock`

Example

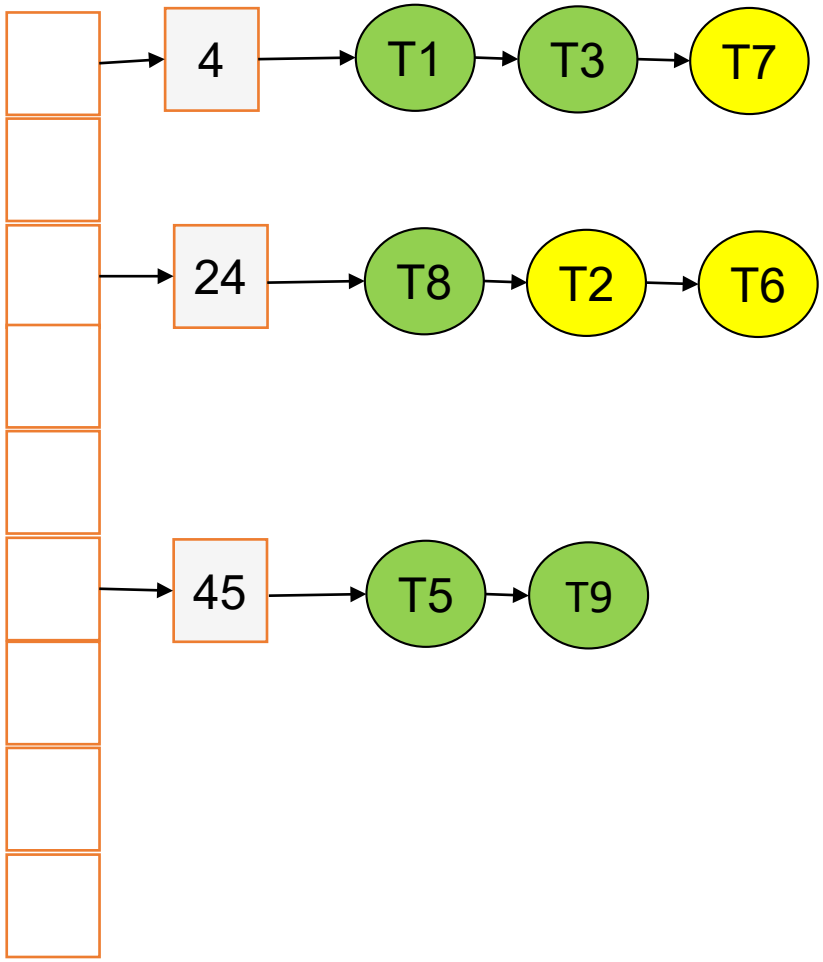
- **Arrival sequence**: r1(x), w1(x), r2(x), r3(y), w1(y), …
- r1(x)
  - r1-lock(x) request → OK →
- w1(x)
  - w1-lock(x) request → OK (upgrade) →
- r2(x)
  - r2-lock(x) request → NO because w-locked
- r3(y)
  - r3-lock(y) request → OK →
  - T3 unlock(y) →
- w1(y)
  - w1-lock(y) request → OK →
  - T1 unlock(x) →
  - T1 unlock(y) →
  - r2(x) was waiting for r2-lock(x) request →
  - …

x state	y state
r-locked, n_x=1	
w-locked	
T2 waits for x	
	r-locked, n_y=1
	free n_y=0
	w-locked
free n_x=0	
	free
r-locked, n_x=1	

Note: T2 is **delayed,** it can be executed when T1 unlocks x

How are locks implemented

- Typically by lock tables, which are hash tables indexing the lockable items via hashing
- Each locked item has a linked list associated with it
- Every node in the linked list represents the transaction which requested for lock, lock mode (SL/XL) and current status (granted/waiting).
- Every new lock request for the data item is appended as a new node to the list.
- Locks can be applied on both data and index records



Is respecting locks enough for serializability?

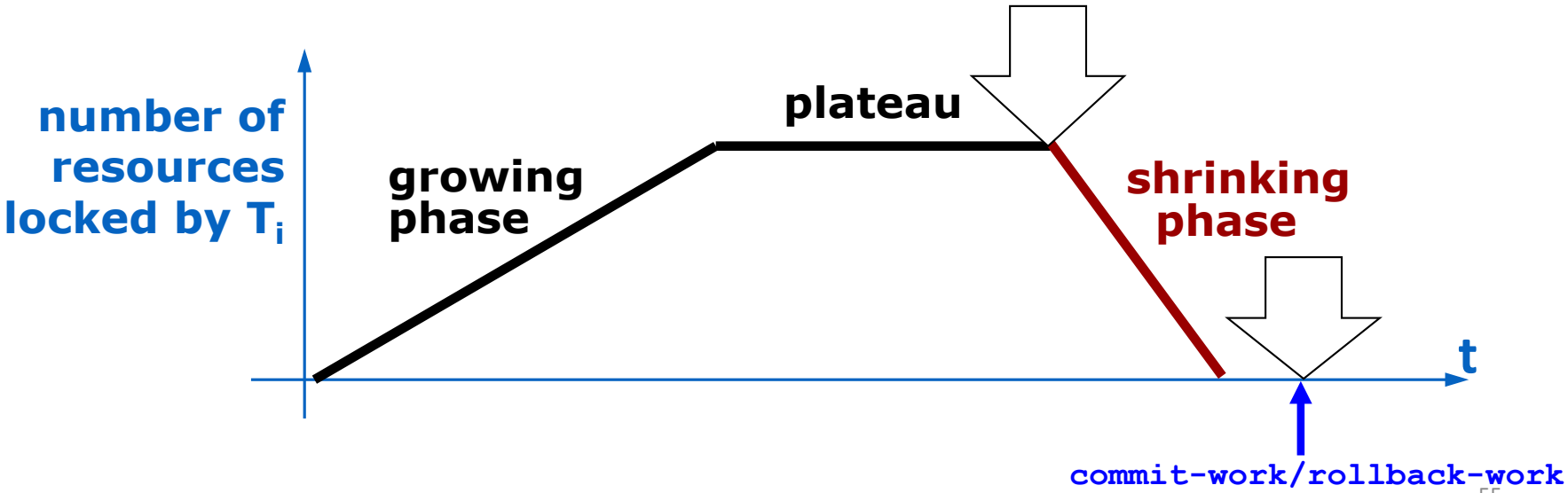
- Arrival sequence r1(x), r2(x), w2(x), r1(x)

		x state
• r1(x)		
• r1-lock(x) request → OK	→	r-locked, n=1
• **T1 unlock(x)**	→	free, n=0
• r2(x)		
• r2-lock(x) request → OK	→	r-locked, n=1
• w2(x)		
• w2-lock(x) request → OK (upgrade)	→	w-locked
• T2 unlock(x)	→	free, n=0
• r1(x)		
• **r1-lock(x)** request → OK	→	r-locked , n=1 ..

- But the schedule mapped by the lock manager does not eliminate T1's **non repeatable read**
- T1 behavior: 1) releases its read lock on x too quickly; 2) acquires another lock after releasing one lock

Two-Phase Locking (2PL)

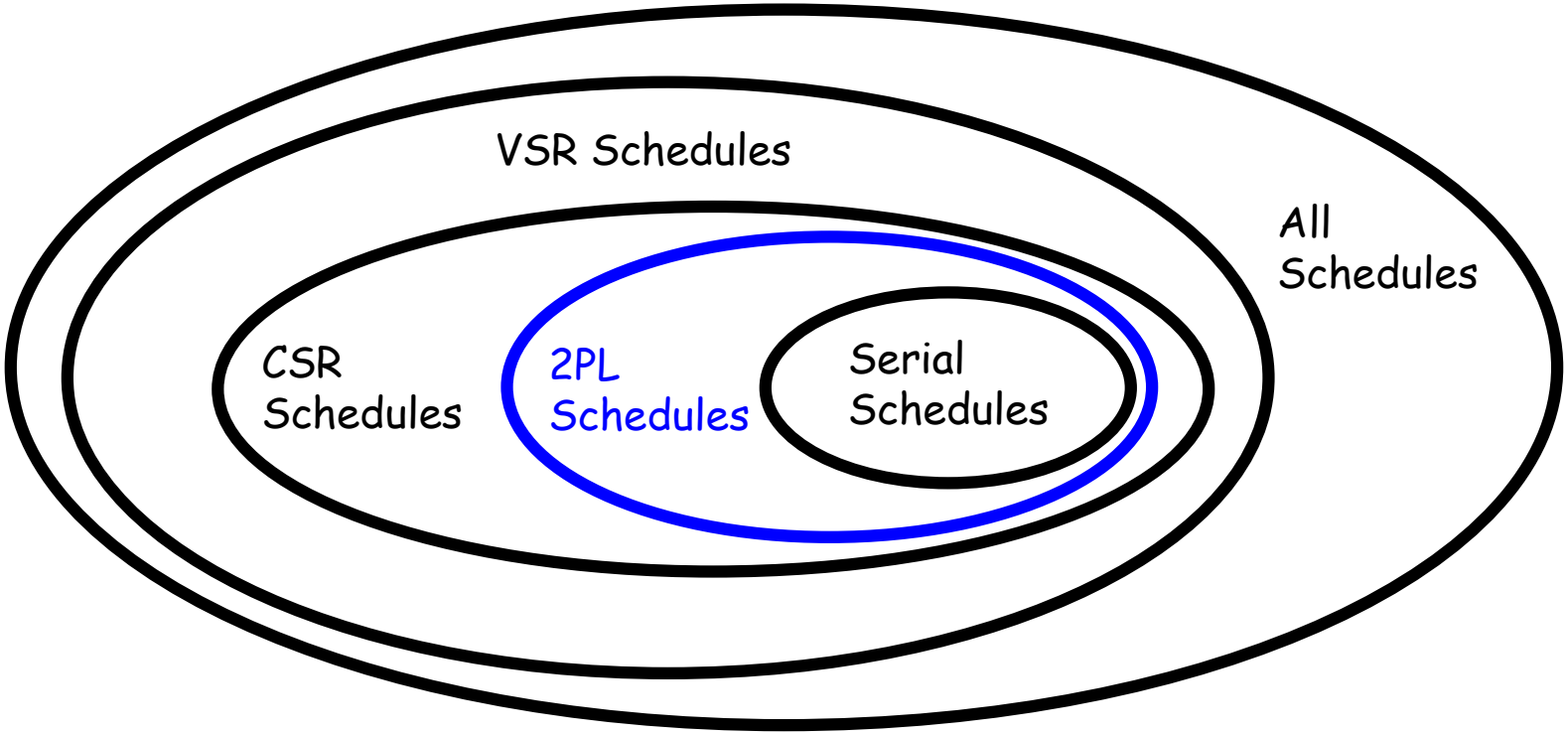
- Requirement (**two-phase rule**):
  - A transaction cannot acquire any other lock after releasing a lock
  - Ensures serializability!



Serializability

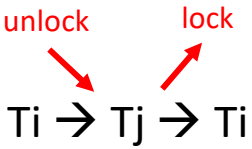
- Consider a scheduler that
  - Only processes well-formed transactions
  - Grants locks according to the conflict table
  - Checks that all transactions apply the two-phase rule
- The class of generated schedules is called **2PL**
- Result: schedules in 2PL are both view- and conflict-serializable
- (VSR $\supset$ CSR $\supset$ 2PL)

CSR, VSR and 2PL



2PL implies CSR

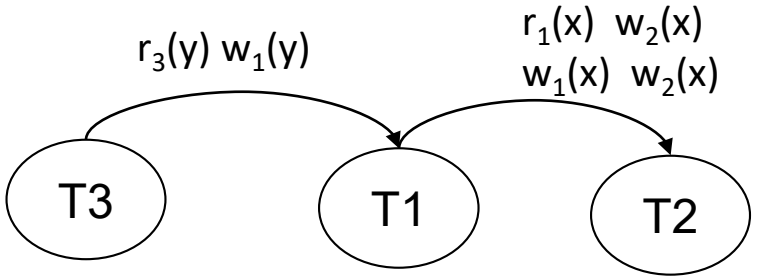
- CSR ⊃ 2PL: Every 2PL schedule is also conflict-serializable
- 2PL ➔ CSR:
  - Suppose a 2PL schedule S is not CSR
    - S's graph must contain at least one cycle Ti → Tj → Ti
    - Therefore there must be a pair of conflicting operations in reverse order, suppose that the conflictual operations appear as follows in the schedule
      - $OP_{hi}(x)$, $OP_{kj}(x)$ … $OP_{uj}(y)$, $OP_{wi}(y)$ where one of $OP_{hi}(x)$, $OP_{kj}(x)$ is a write and one of $OP_{uj}(y)$, $OP_{wi}(y)$ is a write
      - $OP_{hi}(x)$, $OP_{kj}(x)$
        - Ti must have released a lock for Tj to access x
      - Later in the schedule… $OP_{uj}(y)$, $OP_{wi}(y)$
        - Ti must have acquired a lock for the conflict to occur → CONTRADICTION



- Inclusion of 2PL in VSR descends from VSR ⊃ CSR
- This proves that all 2PL schedules are view-serializable too

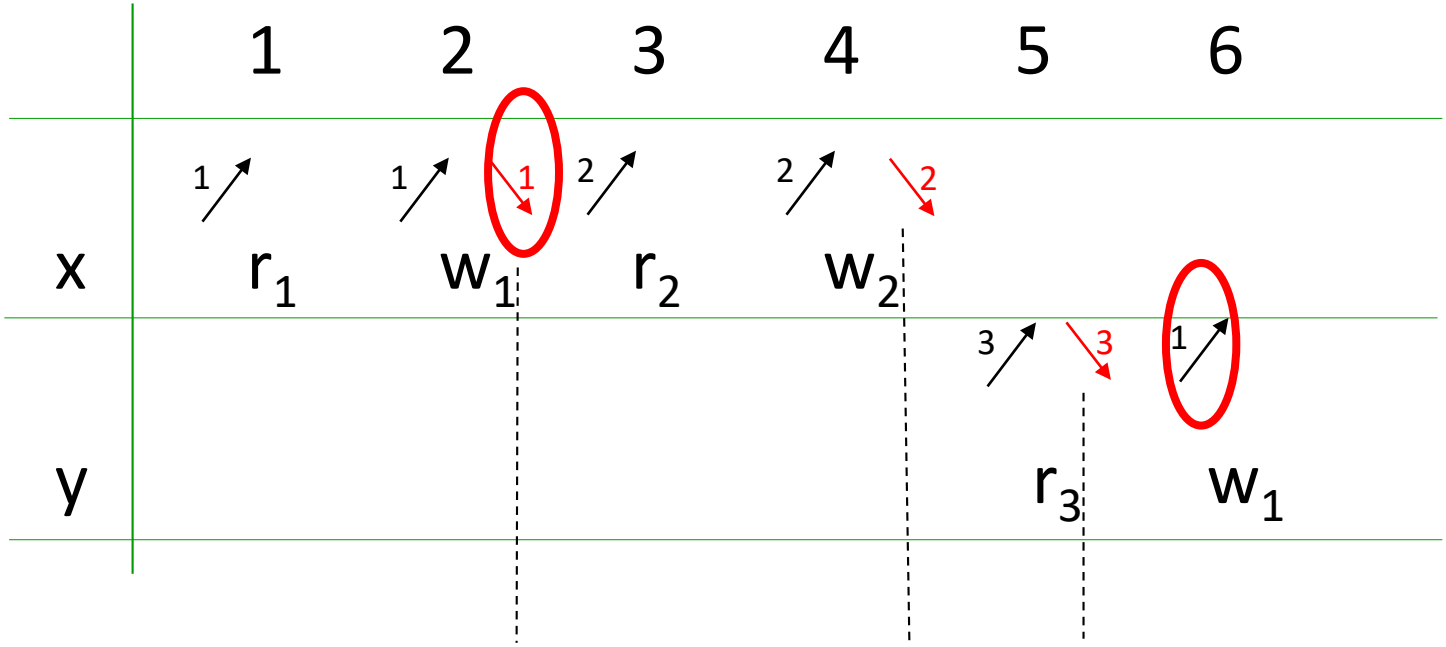
2PL smaller than CSR

- CSR $\supset$ 2PL: Every 2PL schedule is also conflict-serializable, but the converse is not necessarily true
- Counter-example: r1(x) w1(x) r2(x) w2(x) r3(y) w1(y)
  - It violates 2PL
  - r1(x) w1(x) ↘ r2(x) w2(x) r3(y) ↗ w1(y)
  - T1 releases T1 acquires
  - However, it is conflict-serializable: T3 < T1 < T2
    - On x: r1(x) w1(x) r2(x) w2(x)
    - On y: r3(y) w1(y)



A visualization of the 2PL test

Resources on the Y axis and operation times on the X axis



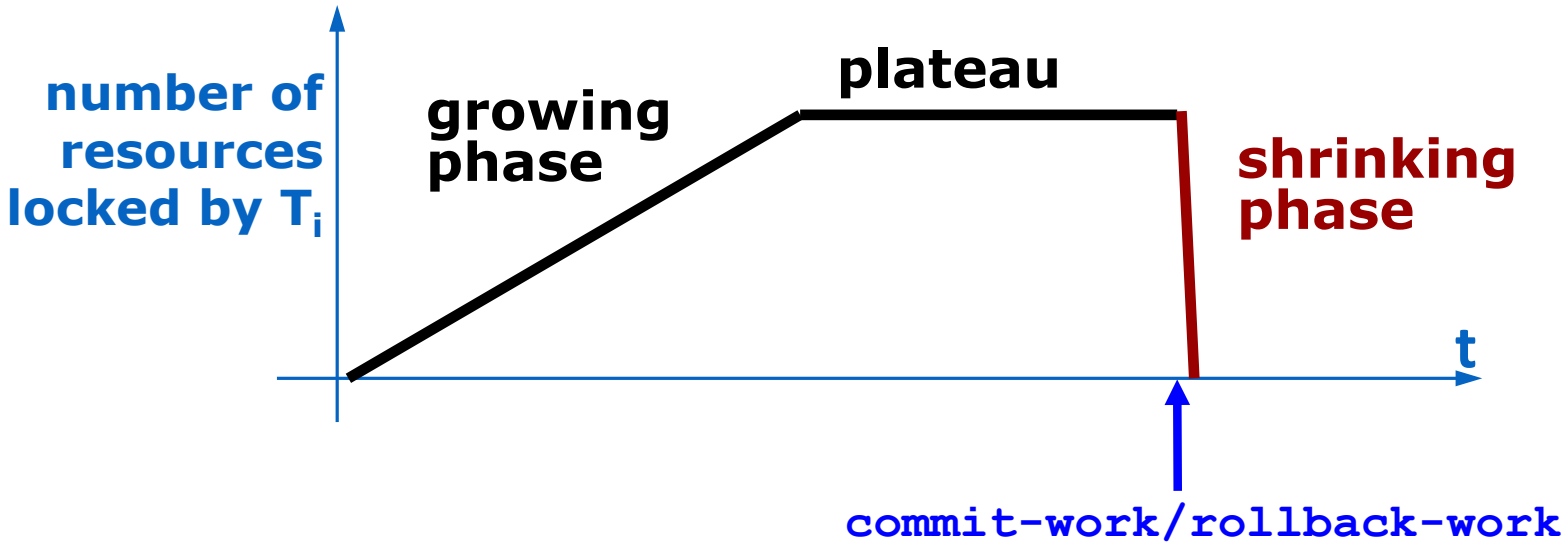
2PL and other anomalies

- Nonrepeatable read r1 - r2 - w2 - r1
  - Already shown
- Lost update r1 - r2 - w2 - w1
  - T1 releases a lock to T2 and then tries to acquire another one
- Phantom update: r1 - r2 - w2 - r1
  - T1 releases a lock to T2 and then tries to acquire another one
- Phantom insert: r1 - w2(new data) - r1
  - T1 releases a lock to T2 and then tries to acquire another one (NOTE: T2 does not necessarily write on data already locked by T1 → **requires** lock on "future data" aka **predicate locks**)
- Dirty read r1 - w1 - r2 - abort1 - w2
  - Requires dealing with abort

Dirty reads are still a menace: Strict 2PL

- Up to now, we were still using the hypothesis of commit-projection (no transactions in the schedule abort)
  - 2PL, as seen so far, does not protect against dirty (uncommitted data) reads (and therefore neither do VSR nor CSR)
    - Releasing locks before rollbacks exposes "dirty" data
- To remove this hypothesis, we need to add a constraint to 2PL, that defines **strict 2PL**:
  - **Locks held by a transaction can be released only after commit/rollback**
  - Remember: rollback restores the state prior to the aborted updates
- This version of 2PL is used in most commercial DBMSs when a high level of isolation is required (see next: SQL isolation levels)

Strict 2PL in Practice



- **Strict 2PL locks** are also called **long duration** locks, **2PL locks** **short duration** locks
- Note: real systems may apply 2PL policies differently to read and write locks
- Typically: long duration strict 2PL write locks, variable policies for read locks
- NOTE: long duration read locks are costly in terms of performances: real systems replace them with more complex mechanisms

How to prevent phantom inserts: predicate locks

- A **phantom insertion** occurs when a transaction adds items to a data set previously read by another transaction
- To prevent phantom inserts a lock should be placed also on "**future data**", i.e., inserted data that would satisfy a previous query
- **Predicate locks** extend the notion of **data locks** to "future data"
- Example:
  - Suppose that transaction T = update Tab set B=1 where A>1
  - Then, the lock is on predicate A>1
  - Other transactions cannot insert, delete, or update any tuple satisfying this predicate
- In the worst case (predicate locks not supported):
  - The lock extends to the entire table
- In case the implementation supports predicate locks:
  - The lock is managed with the help of indexes (gap lock)

Tab

A	B

Example:
T_1: C=AVG(B: A=1)
T_2: Insert (A=1,B=2)
T_1: C=AVG(B: A=1)

For T_1 → lock on predicate A=1
→ T_2 cannot Insert (A=1,B=2)

Isolation Levels in SQL:1999 (and JDBC)

- SQL defines transaction isolation levels which specify the anomalies that should be prevented by running at that level
- The level does not affect write locks. A transaction should always get exclusive lock on any data it modifies, and hold it until completion (**strict 2PL on write locks**), regardless of the isolation level. For read operations, levels define the degree of protection from the effects of modifications made by other transactions

Why long duration write locks are necessary

- Consider the following schedule (admissible if write locks are short duration) and **remove the hypothesis that aborts do not occur**
- w1[x] ... w2[x]...((c1 or **a1**) and (c2 or **a2**) in any order)
  - **T2** is allowed to write over the same object updated by **T1** which has not yet completed
- If T1 aborts… e.g., w1[x]...w2[x] ... , **a1**, (c2 or a2)
  - How to process event a1?
    - If x is restored to the state before T1, T2's update is lost, so if T2 commits x has a stale value
    - If x is NOT restored and T2 also aborts… then T2's proper before state cannot be reinstalled either!
- Thus: write locks are held until the completion of the transaction to enable the proper processing of abort events
- The anomaly of the above non commit-projection schedule is named **dirty write**

Isolation Levels in SQL:1999 (and JDBC)

- READ UNCOMMITTED allows dirty reads, nonrepeatable reads and phantom updates and inserts:
  - **No read locks** (and ignores locks of other transactions)
- READ COMMITTED prevents dirty reads but allows nonrepeatable reads and phantom updates/inserts:
  - **Read locks** (and complies with locks of other transactions), but **without 2PL** on read locks (*read locks are released as soon as the read operation is performed and can be acquired again*)
- REPEATABLE READ avoids dirty reads, nonrepeatable reads and phantom updates, but allows phantom inserts:
  - long duration read locks → **2PL also for reads**
- SERIALIZABLE avoids all anomalies:
  - **2PL with predicate locks** to avoid phantom inserts
- Note that SQL standard isolation levels dictate minimum requirements, real systems may go beyond (e.g., in MySQL and Postgres REPEATABLE READ avoids phantom inserts too) and use different mechanisms (e.g., to avoid long duration read locks)

	Dirty Read	Non rep. read	Phantoms
Read uncommitted	Y	Y	Y
Read committed	N	Y	Y
Repeatable read	N	N	Y (insert)
Serializable	N	N	N

SQL92 serializable <> serial !

- Serializable transactions don't necessarily execute serially
- The requirement is that transactions can only commit if the result would be as if they had executed serially in some order
- The locking requirements to meet this guarantee can frequently lead to a deadlock (see next slides) where one of the transactions needs to be rolled back
- Therefore, the SERIALIZABLE isolation level is used sparingly and is NOT the default in most commercial systems

SQL isolation levels and locks

- SQL Isolation levels may be implemented with the appropriate use of locks
- Commercial systems make joint use of locks and of timestamp-based concurrency control mechanisms

	READ LOCKS	WRITE LOCKS
READ UNCOMMITTED	Not required	Well formed writes Long duration write locks
READ COMMITTED	Well formed reads Short duration read locks (data and predicate)	Well formed writes Long duration write locks
REPEATABLE READ	Well formed reads Long duration data read locks Short duration predicate read locks	Well formed writes Long duration write locks
SERIALIZABLE	Well formed reads Long duration read locks (predicate and data)	Well formed writes Long duration write locks

Setting transaction characteristics in SQL

```
<set transaction statement> ::=
SET [ LOCAL ] TRANSACTION <transaction characteristics>

<transaction characteristics> ::= [ <transaction mode> [ { <comma>
  <transaction mode> }... ] ]

<transaction mode> ::= <isolation level> | <transaction access
mode>
  | <diagnostics size>

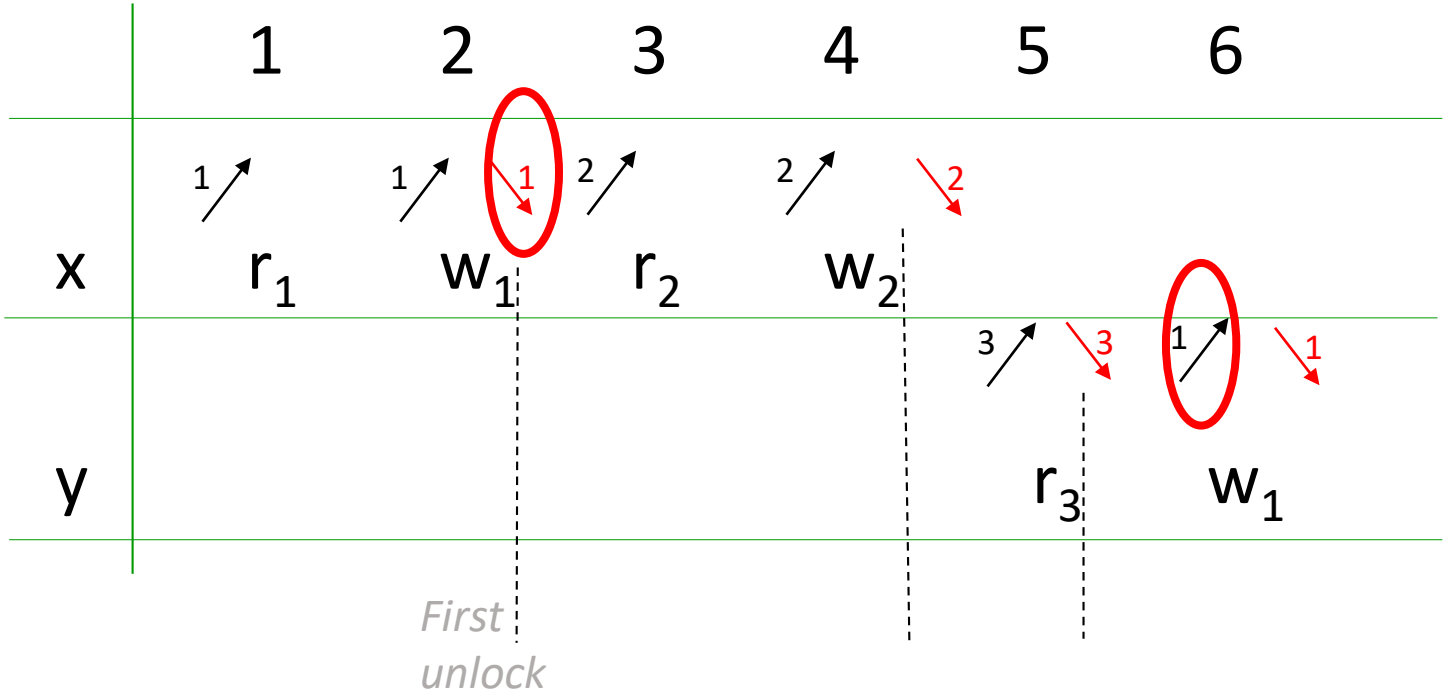
<transaction access mode> ::= READ ONLY| READ WRITE

<isolation level> ::= ISOLATION LEVEL <level of isolation>

<level of isolation> ::= READ UNCOMMITTED | READ COMMITTED
  | REPEATABLE READ | SERIALIZABLE
```

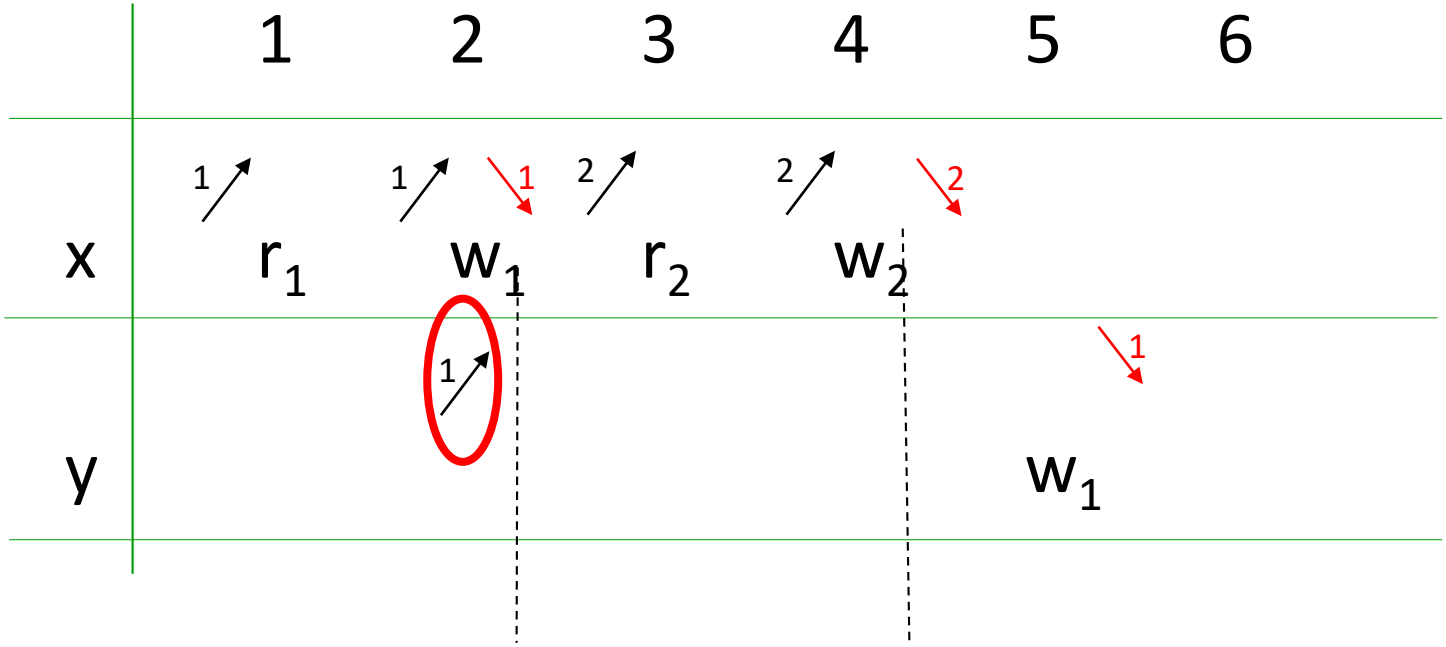
A visualization of the 2PL test

Resources on the Y axis and operation times on the X axis



2PL test = can we arrange read/write lock + unlock requests so that the 2PL rule is satisfied for the given schedule?

Let's remove r3 from the schedule

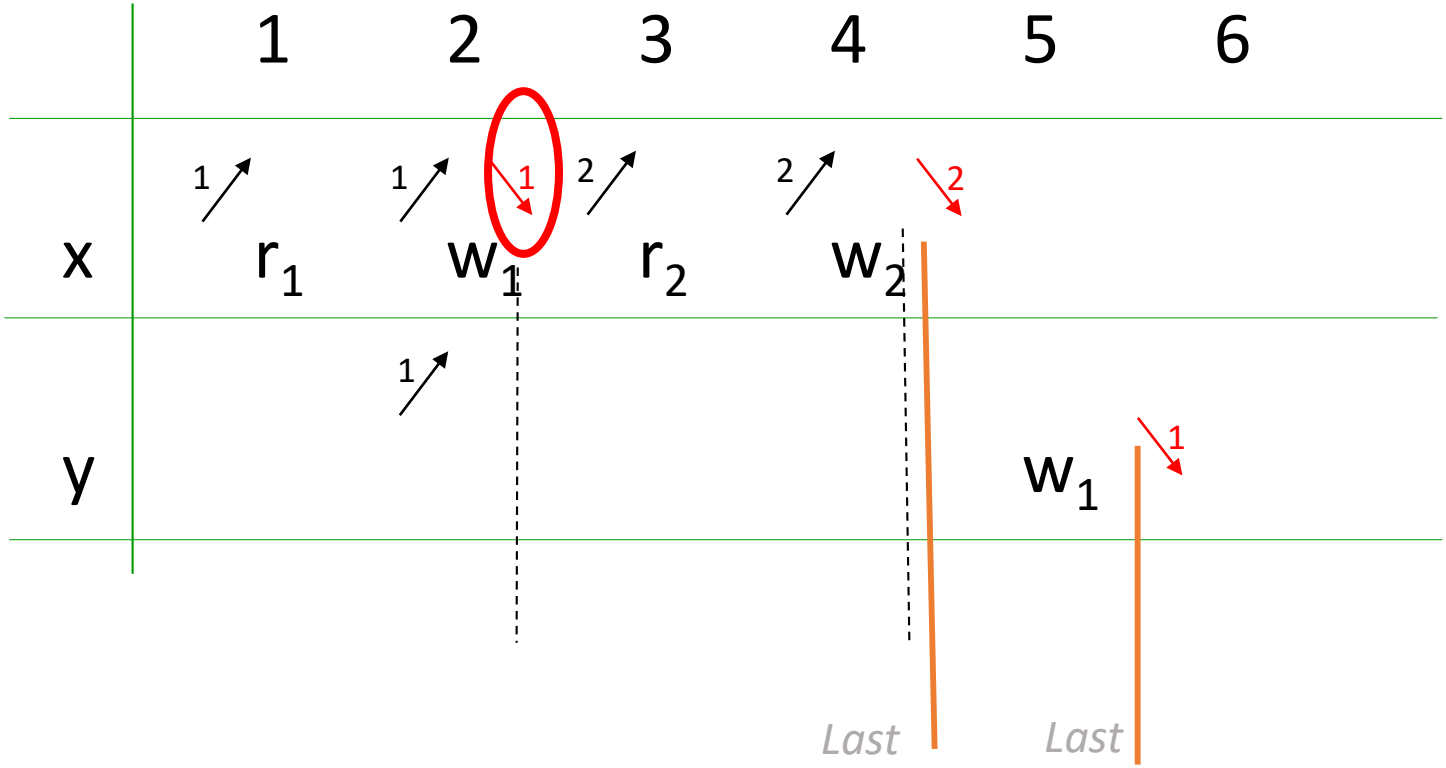


And now? Can we arrange read/write lock + unlock requests so that the 2PL rule is satisfied for this new schedule?

Is it 2PL strict?

Let's remove r3 from the schedule

Is this schedule strict 2PL?



No, it is not strict 2PL because T1 needs to Release a lock before the commit (last operation)

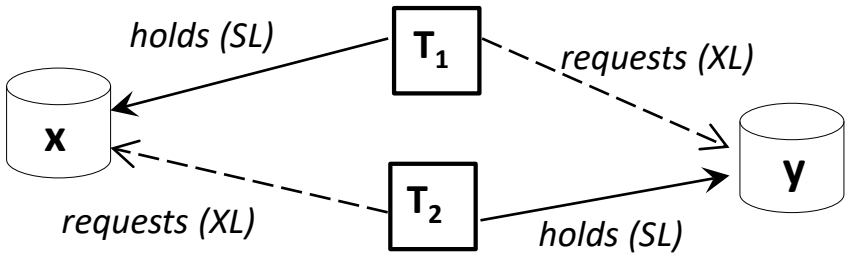
The impact of locking: waiting is dangerous!

- Transactions requesting locks are either **granted the lock** or **suspended and queued** (first-in first-out). There is risk of:
  - **Deadlock**: two or more transactions in endless (mutual) wait
    - Typically occurs when each transaction waits for another to release a lock (in particular: r1 r2 w1 w2 – see next slide)
  - **Starvation**: a single transaction in endless wait
    - Typically occurs due to write transactions waiting for resources that are continuously read (e.g., index roots)

Deadlock

- Occurs because concurrent transactions hold and in turn request resources held by other transactions

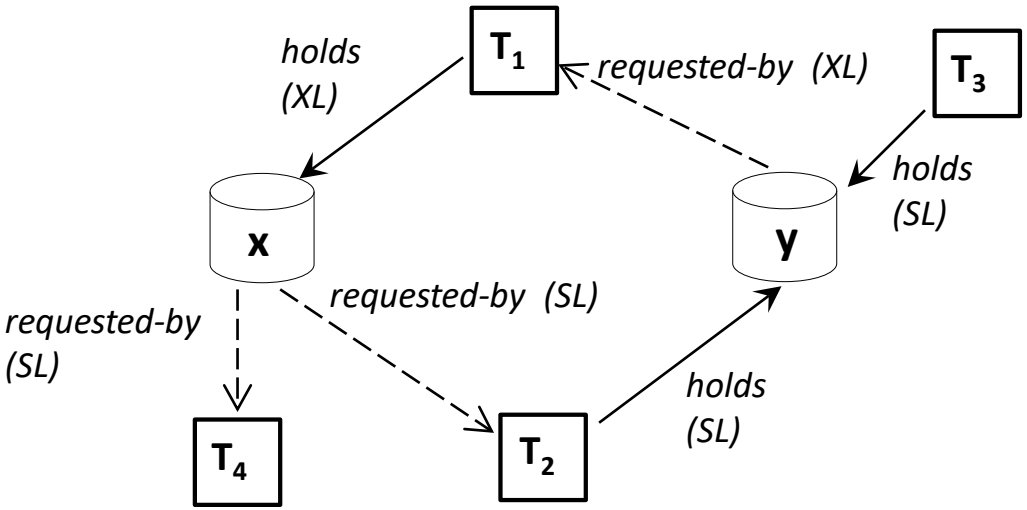
- T1 : r1(x) w1(y)
- T2 : r2(y) w2(x)

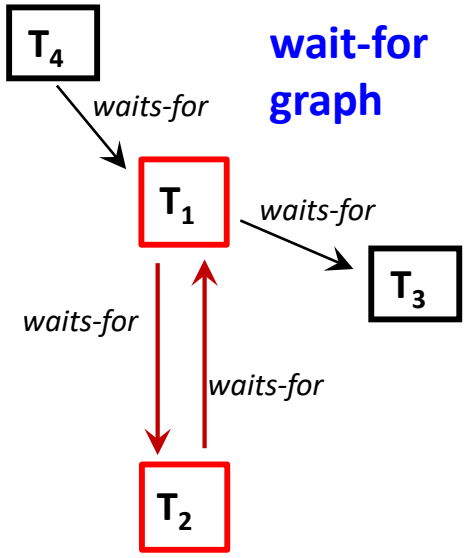


- S: r_lock1(x),
  r_lock2(y),
  r1(x), r2(y),
  w_lock1(y), [wait!]
  w_lock2(x) [wait!]→ deadlock

Deadlock

- Lock graph: a bipartite graph in which nodes are resources or transactions and arcs are lock requests or lock assignments
- Wait-for graph: a graph in which nodes are transactions and arcs are "waits for" relationships
- A deadlock is represented by a cycle in the wait-for graph of transactions





Deadlock Resolution Techniques

- Timeout
  - Transactions killed after a long wait
    - How long?
- Deadlock prevention
  - Transactions killed when they COULD BE in a deadlock
    - Heuristics
- Deadlock detection
  - Transactions killed when they ARE in a deadlock
    - Inspection of the wait-for graph

1) Timeout Method

- A transaction is killed and restarted after a given amount of waiting (assumed as due to a deadlock)
  - The simplest method, widely used in the past
- The timeout value is system-determined (sometimes it can be altered by the database administrator)
- The problem is choosing a proper timeout value
  - Too long: useless waits whenever deadlocks occur
  - Too short: unrequired kills, redo overhead

2) Deadlock Prevention

- Idea: killing transactions that could cause cycles
- **Resource-based prevention**: restrictions on lock requests
  - Transactions request all resources at once, and only once
  - Resources are globally sorted and must be requested "in global order"
  - Problem: it's not easy for transactions to anticipate all requests!
- **Transaction-based prevention**: restrictions based on transactions' IDs
  - Assigning IDs to transactions incrementally → transactions' "age"
  - Preventing "older" transactions from waiting for "younger" ones to end their work
  - Options for choosing the transaction to kill
    - **Preemptive** (killing the holding transaction – wound-wait)
    - **Non-preemptive** (killing the requesting transaction – wait-die)
  - Problem: too many "killings"! (waiting probability >> deadlock probability)

Non-preemptive algorithm: Wait-Die

Conflicts are between a **requesting transaction (RT)** and a **conflicting transaction (CT), that holds the lock**.

Wait-Die Algorithm:
If RT T1 is older than CT T2, then T1 waits, otherwise T1 dies.
This is a non-preemptive algorithm in which RT never forces CT to abort.



It is restarted with the same time stamp (it becomes older!) – avoids starvation

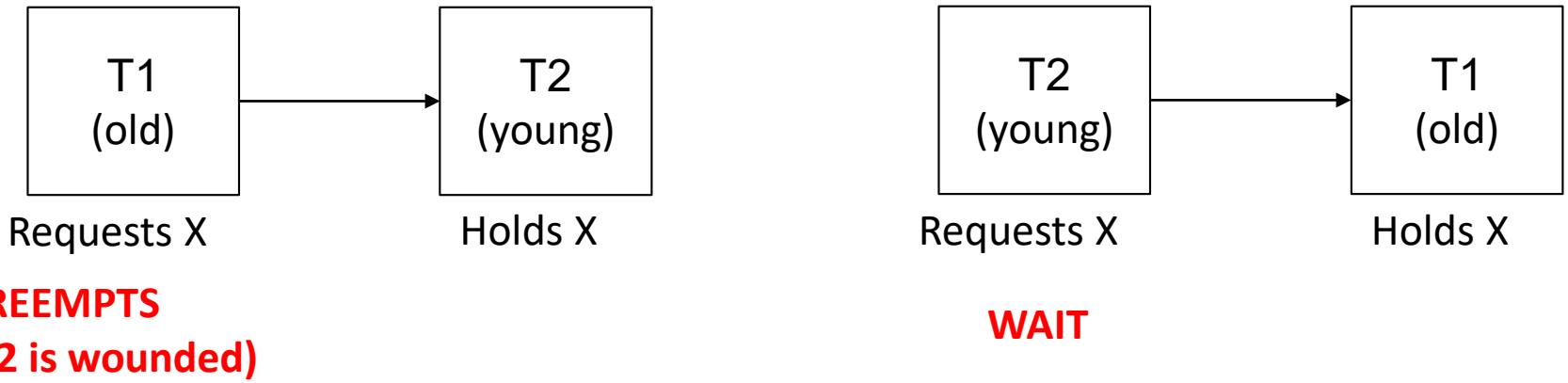
The oldest transaction survives!

The young transaction is killed when it **(the young transaction) requests** a lock held by the older transaction

Preemptive algorithm: Wound-Wait

- **Wound-Wait Algorithm:**

If RT T1 is older than CT T2, then T1 wounds T2, otherwise T1 waits.
This is a preemptive algorithm.



The oldest transaction survives!

The young transaction is killed when **the old transaction requests a lock** held by the young transaction

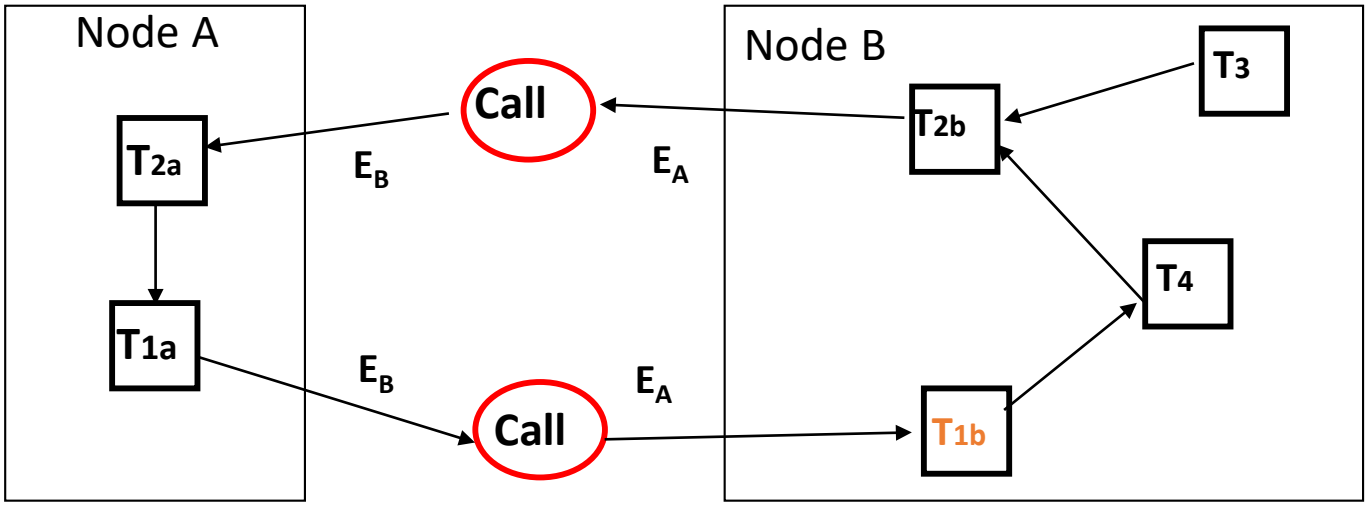
3) Deadlock Detection

- Requires an algorithm to detect cycles in the wait-for graph
  - http://davebland.com/how-often-does-sql-server-look-for-deadlocks
  - Must work with **distributed** resources efficiently & reliably

- An elegant solution: **Obermarck's** algorithm (DB2-IBM, published on ACM Transactions on Database Systems)
- Assumptions
  - Transactions execute on a single main node (one locus of control)
  - Transactions may be decomposed in "sub-transactions" running on other nodes
  - Synchronicity: when a transaction spawns a sub-transaction it suspends work until the latter completes
  - All transactions must have a single originating point, i.e., there must be a single sub-transaction that is not waiting for any other sub-transaction of the same transaction
  - Two wait-for relationships:
    - T_i waits for T_j on the same node because Ti needs a datum locked by T_j
    - A sub-transaction of T_i waits for another sub-transaction of T_i running on a different node (via external call E)

Distributed Deadlock Detection

Distributed dependency graph: external call nodes represent a sub-transaction activating another sub-transaction at a different node

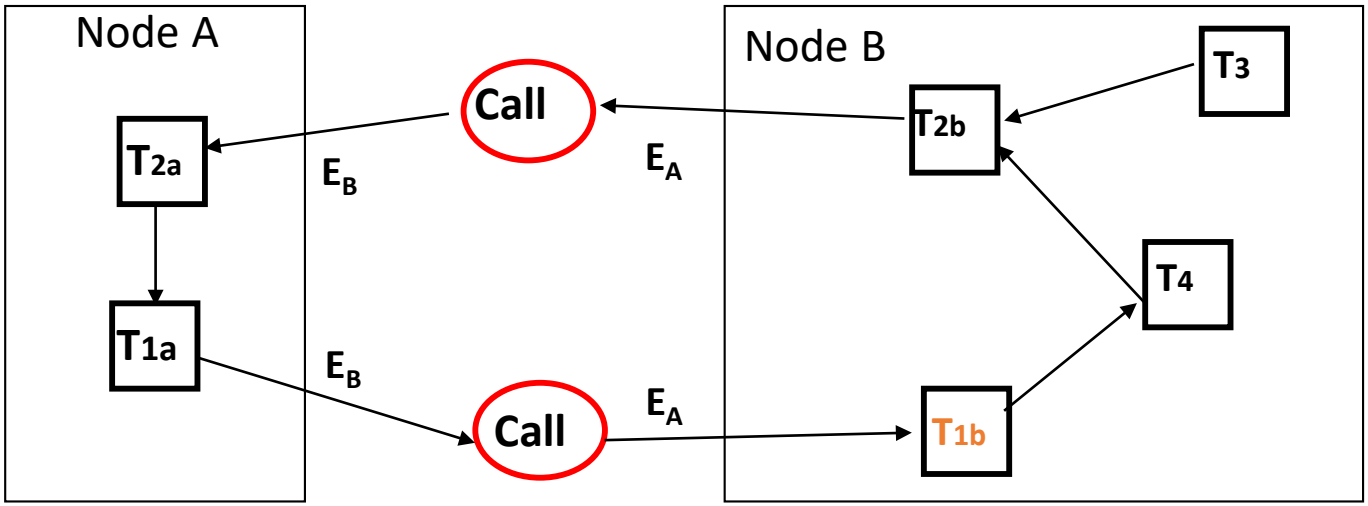


- Representation of the status
  - at Node A: $E_b \rightarrow T_{2a} \rightarrow T_{1a} \rightarrow E_b$
  - at Node B: $E_a \rightarrow T_{1b} \rightarrow T_{2b} \rightarrow E_a$
- NOTATION: symbol → : 1) "wait for" relation among local transactions 2) if one term is an external call, either the source is waited for by a remote transaction or the sink waits for a remote transaction
- Potential Deadlock: T_{2a} waits for T_{1a} (data lock) that waits for T_{1b} (call) that waits for T_{2b} (data locks) that waits for T_{2a} (call): cycle!
- Problem: how to detect such an occurrence <u>without maintaining the global view</u>

Obermarck's Algorithm

- Goal: detection of a potential deadlock looking only at the local view of a node
- Method: establishing a communication protocol whereby each node has a local projection of the global dependencies
- Nodes exchange information and update their local graph based on the received information
- Communication is optimized to avoid that multiple nodes detect the same potential deadlock
- Node A sends its local info to a node B only if
  - A contains a transaction T_i that is waited for from another remote transaction and waits for a transaction T_j in turn waiting for another remote transaction on B
  - i>j (this ensure a kind of message "forwarding" along a node path where node A "precedes" node B if i>j)
  - Mnemonically: I send info to you if a distributed transaction listed at me waits for a distributed transaction listed at you with **smaller** index

Forwarding rule



- Node A:
  - Activation/wait sequence: $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$
    - i=2, j=1
    - A can dispatch info to B

i > j ✓

- Node B:
  - Activation/wait sequence: $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$
    - i=1, j=2
    - B does not dispatch info to A

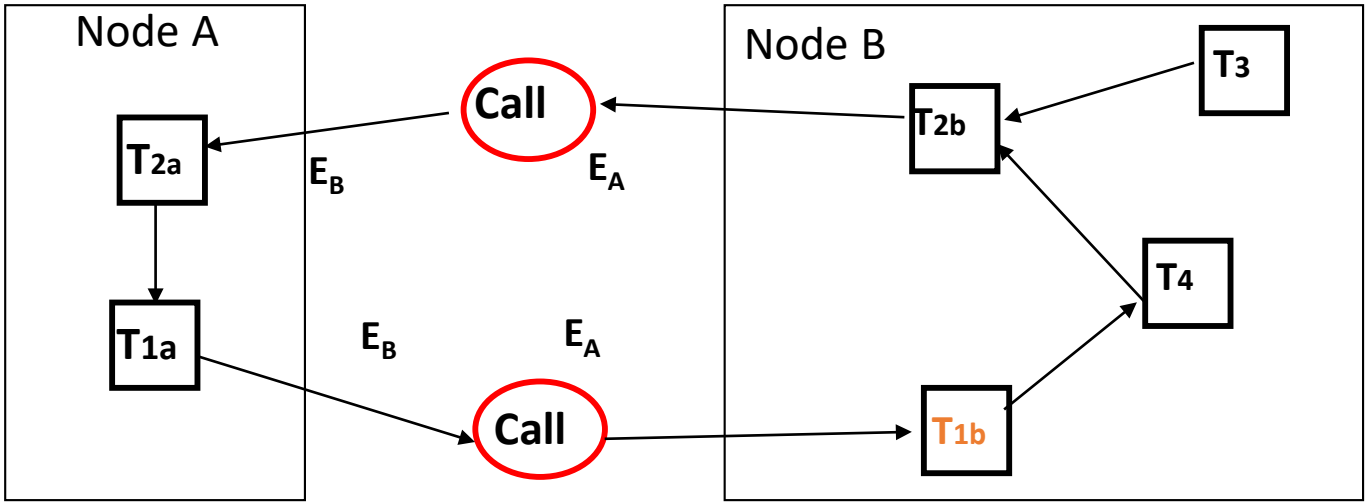
(only distributed transactions count)

i < j

Obermarck's Algorithm

- Runs periodically at each node
- Consists of 4 steps
  - Get graph info (wait dependencies among transactions and external calls) from the "previous" nodes. Sequences contain only node and top-level transaction identifiers
  - Update the local graph by merging the received information
  - Check the existence of cycles among transactions denoting potential deadlocks: if found, select one transaction in the cycle and kill it
  - Send updated graph info to the "next" nodes
  - Propagate also killed transactions

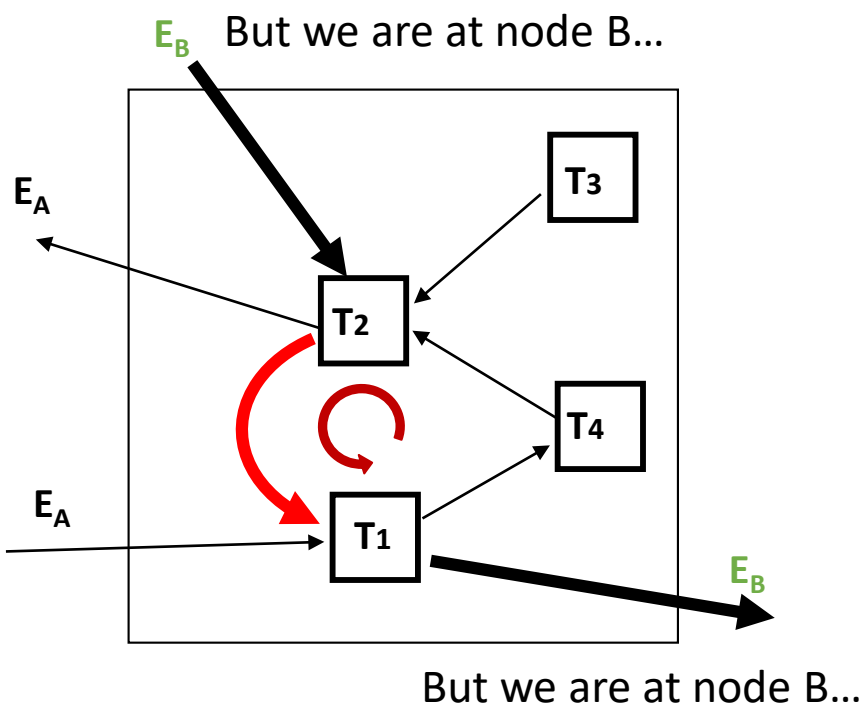
Algorithm execution, step 1: communication



- Distributed Deadlock Detection: forwarding rule
  - at Node A: $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ info sent to Node B
  - at Node B: $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$ info not sent (i<j)

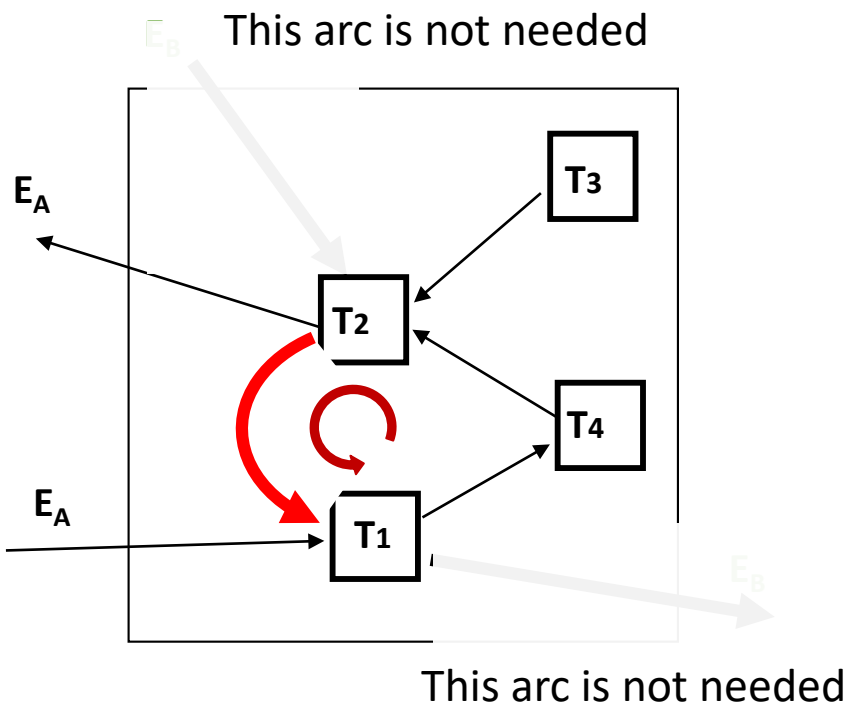
Algorithm execution, step 2: local graph update

- at Node B:
- 1. Eb → T2 → T1 → Eb is received
- 2. Eb → T2 → T1 → Eb is added to the local wait-for graph

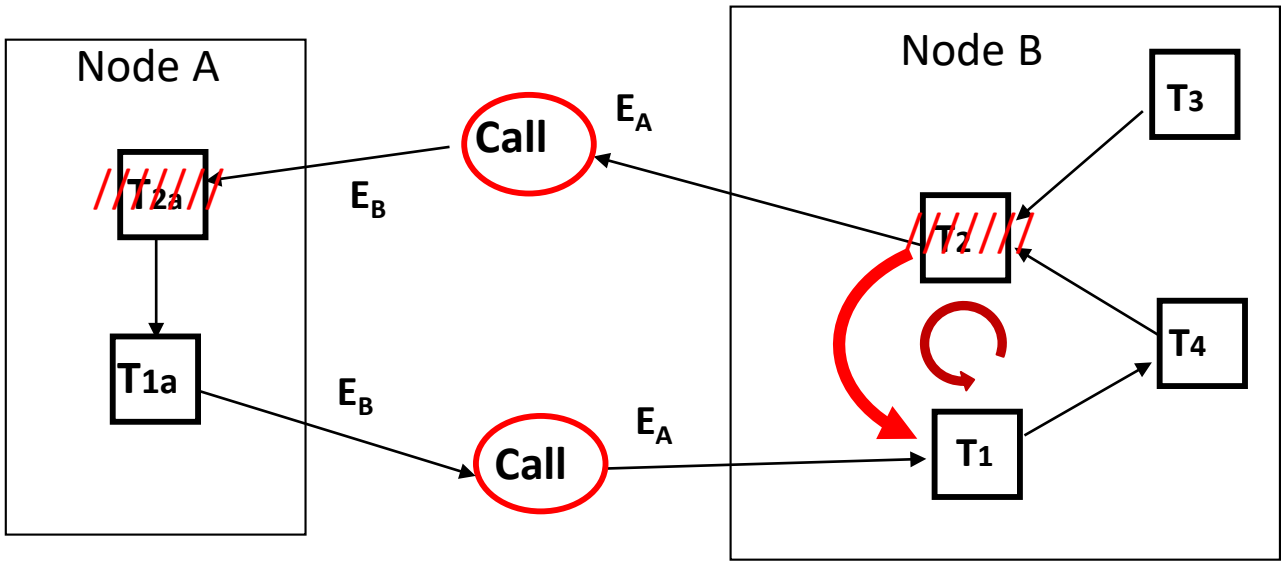


Algorithm execution, step 2: local graph update

- at Node B:
- 1. Eb → T2 → T1 → Eb is received
- 2. Eb → T2 → T1 → Eb is added to the local wait-for graph
- 3. Deadlock detected (cycle among T1 and T2:
- T1 or T2 (or T4) killed (rollback)



Algorithm execution: deadlock resolution



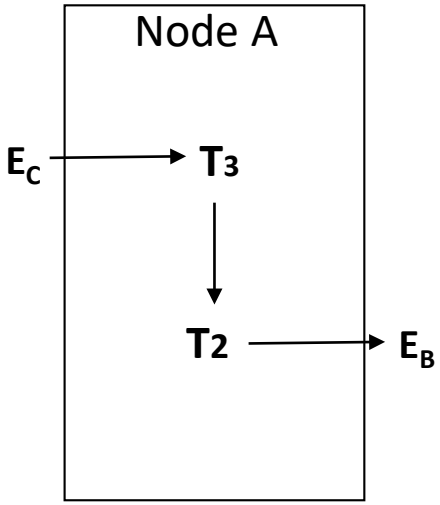
- For example, T2 killed
- Forward info about killed transaction to node A

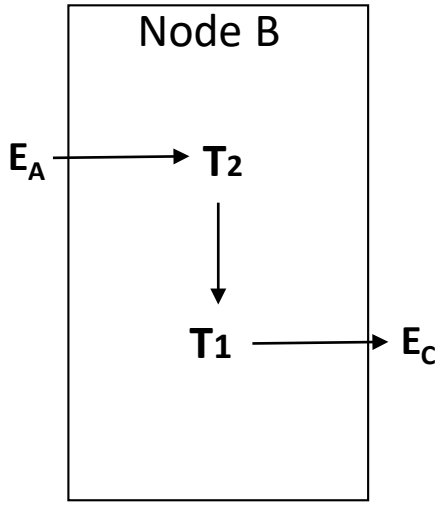
Obermarck algorithm (original paper - 1982)

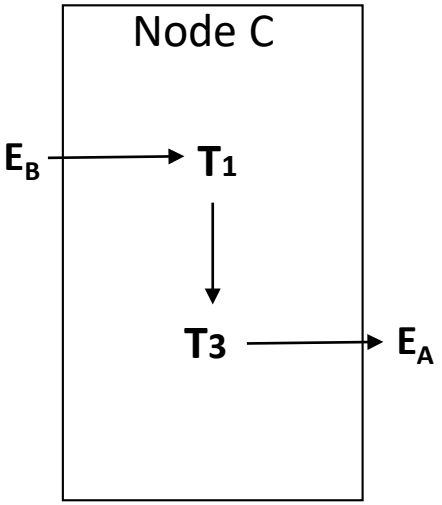
Another example

- Initially: at Node A, $E_C \rightarrow T3 \rightarrow T2 \rightarrow E_B$
  at Node B, $E_A \rightarrow T2 \rightarrow T1 \rightarrow E_C$
  at Node C, $E_B \rightarrow T1 \rightarrow T3 \rightarrow E_A$

Another example, continued





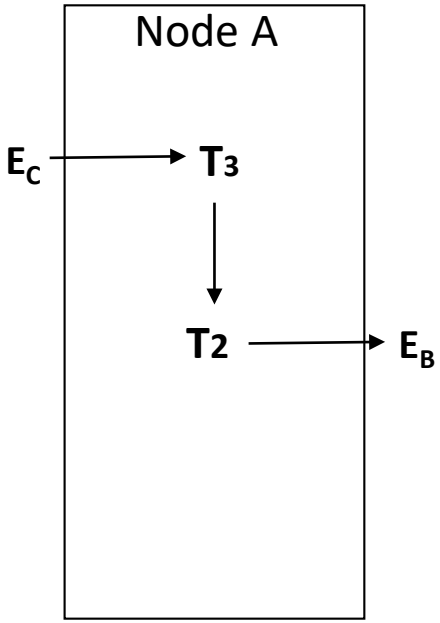


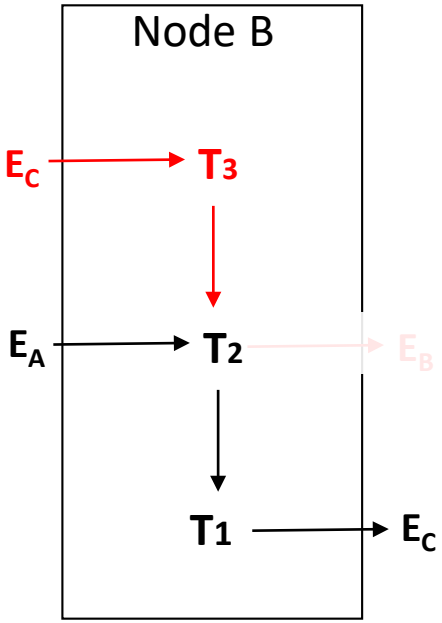
3 > 2 can send (to B)

2 > 1 can send (to C)

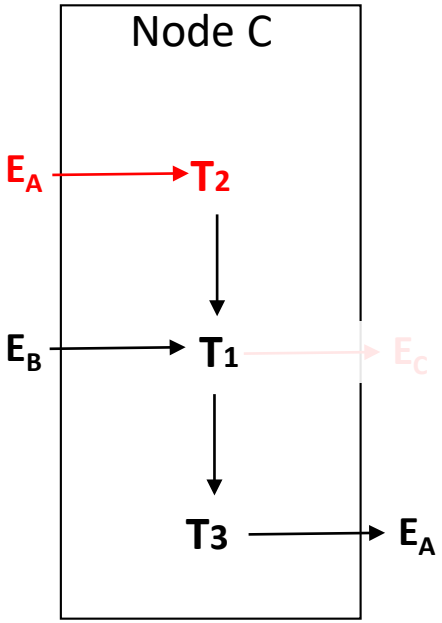
1 < 3 cannot send

Another example, continued



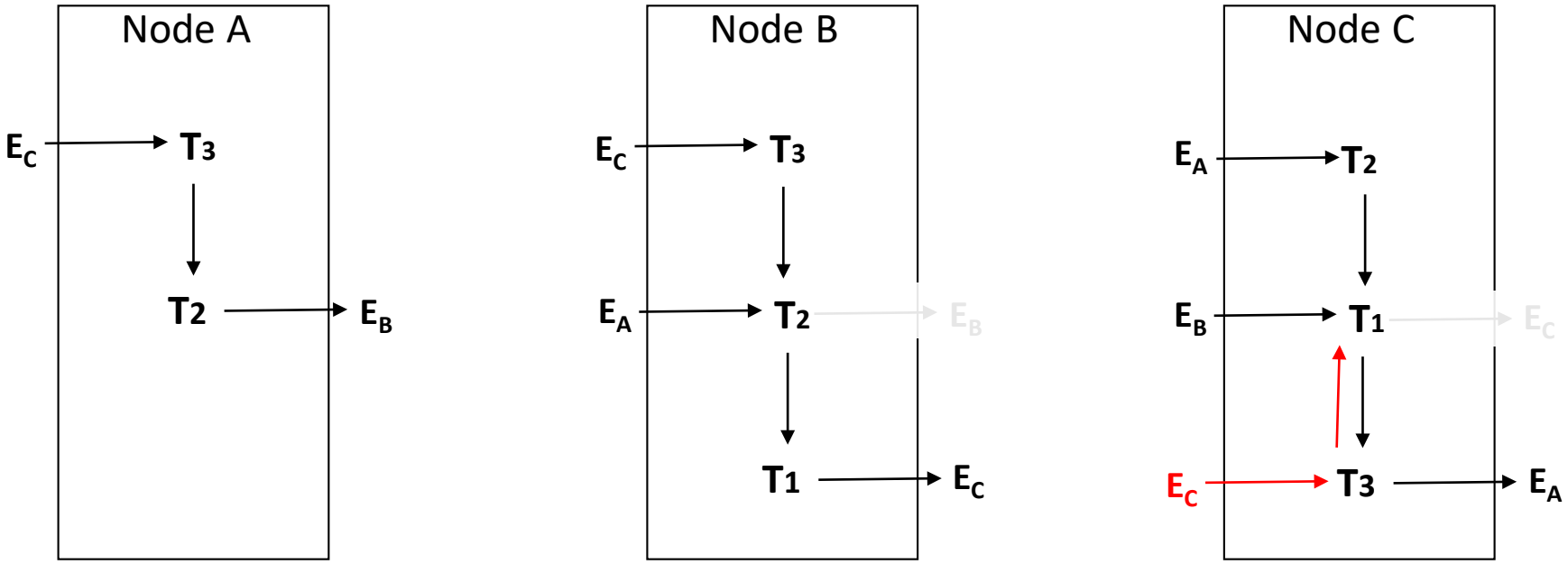


3 > 1 can send (to C)
Ec →T3 →T1 → Ec



2 < 3 cannot send

Another example, continued



Cycle detected!

Obermarck: immateriality of conventions

- There are two arbitrary choices in the algorithm:
- Send messages only if: (1) i > j vs. (2) i < j
- Send them to: (a) the following node vs. (b) the preceding node
- Therefore, there are four versions/variants of the algorithm
- (1+a), (1+b), (2+a), (2+b)
  - The sequence of the sent messages is different
  - However, they all identify deadlocks (if present)

Deadlocks in practice

- Their probability is much less than the conflict probability
  - Consider a file with n records and two transactions doing two accesses to their records (uniform distribution); then:
    - Conflict probability is O(1/n): $\sum_{i=1..n} (1/n * 1/n) = n * (1/n * 1/n) = 1/n$
    - Deadlock probability is $O(1/n^2)$: T1 conflicts with T2 AND vice versa
- Still, they do occur (once every minute in a mid-size bank)
  - The probability is linear in the number of transactions, quadratic in their length (measured by the number of lock requests)
    - Shorter transactions are healthier (ceteris paribus)
- There are techniques to limit the frequency of deadlocks
  - Update Lock, Hierarchical Lock, …,

Update lock

- The most frequent deadlock occurs when 2 concurrent transactions start by reading the same resources (SL) and then decide to write and try to upgrade their lock to XL
- To avoid this situation, systems offer the UPDATE LOCK (UL) – asked by transactions that will **read and then write**

	Resource status			
Request	free	SL	UL	XL
SL	OK	OK	**OK**	No
UL	OK	**OK**	**No**	No
XL	OK	No	No	No

- Update locks are easy to implement and mitigate the most frequent cause of collision: r1(x) r2(x) w1(x) w2(x)

Update lock

r1(x) r2(x) w1(x) w2(x)

Locks for r1(x)w1(x)
SL→(upgrade)XL

Sequence of lock requests with SL and XL only:

SL1 (granted), SL2 (granted), XL1 (T1 waits) , XL2 (T2 waits)

Deadlock!

	Resource status			
Request	free	SL	UL	XL
SL	OK	OK	**OK**	No
UL	OK	**OK**	**No**	No
XL	OK	No	No	No

Locks for r1(x)w1(x)
UL→(upgrade)XL

Sequence of lock requests with UL:

UL1 (granted), UL2 (T2 waits), XL1 (granted), …

No deadlocks!

Avoiding lock-upgrade deadlocks in DBMSs

Problem: Two transactions read the same row and then both try to update it

SQL Server: explicit update lock

```
BEGIN TRANSACTION;
SELECT *
  FROM Event WITH (UPDLOCK)
  WHERE event_id = 12;
UPDATE Event
  SET available_seats = available_seats – 1
  WHERE event_id = 12;
COMMIT;
```

UPDLOCK acquires an update lock (UL)

Avoiding lock-upgrade deadlocks in DBMSs

Problem: Two transactions read the same row and then both try to update it

PostgreSQL: stronger row lock acquired at read time

```
BEGIN;
SELECT *
   FROM Event
   WHERE event_id = 12
   FOR UPDATE;
UPDATE Event
   SET available_seats = available_seats – 1
   WHERE event_id = 12;
COMMIT;
```

FOR UPDATE locks the row before the update. Conceptually: acquire an XL-like lock instead of first acquiring SL and then upgrading it

The syntax and lock modes are DBMS-specific, but the goal is the same: prevent competing lock upgrades

SELECT statement in MySQL

```
SELECT
    [ALL | DISTINCT | DISTINCTROW ]
    [HIGH_PRIORITY]
    [STRAIGHT_JOIN]
    [SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
    [SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
    select_expr [, select_expr] ...
    [into_option]
    [FROM table_references
      [PARTITION partition_list]]
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position}, ... [WITH ROLLUP]]
    [HAVING where_condition]
    [WINDOW window_name AS (window_spec)
        [, window_name AS (window_spec)] ...]
    [ORDER BY {col_name | expr | position}
      [ASC | DESC], ... [WITH ROLLUP]]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
    [into_option]
    [FOR {UPDATE | SHARE}
        [OF tbl_name [, tbl_name] ...]
        [NOWAIT | SKIP LOCKED]
      | LOCK IN SHARE MODE]
    [into_option]
```

FOR UPDATE: Rows examined by the query are write-locked until the end of the current transaction

FOR SHARE: other transactions can read the examined rows but cannot update or delete them

NOWAIT causes a FOR UPDATE or FOR SHARE query to execute immediately, returning an error if a row lock cannot be obtained due to a lock held by another transaction

SKIP LOCKED causes a FOR UPDATE or FOR SHARE query to execute immediately, excluding rows from the result set that are locked by another transaction.

SETUP

```
CREATE TABLE t (i INT, PRIMARY KEY (i)) ENGINE = InnoDB;
INTO t (i) VALUES (1),(2),(3);
```

T1 – lock row 2

```
START TRANSACTION;

SELECT * FROM t
WHERE i = 2
FOR UPDATE
```

Result → 2

Row 2 is locked until COMMIT or ROLLBACK

… still running

T2 – do not wait

```
START TRANSACTION;

SELECT * FROM t
WHERE i = 2
FOR UPDATE NOWAIT;
```

Result → ERROR 3572

Do not wait for lock!
Execute a command only if it doesn't run into a conflict

T3 – skip locked row

```
START TRANSACTION;

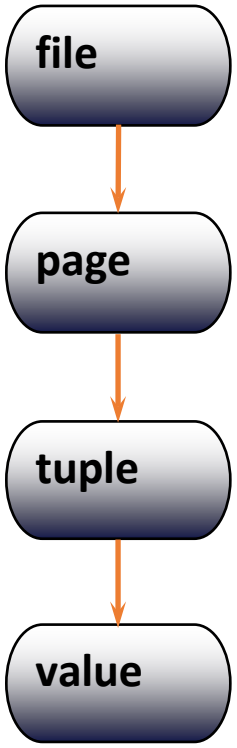
SELECT * FROM t
FOR UPDATE SKIP LOCKED;
```

Result → 1, 3

You get unclaimed records – only committed records
→ when avoiding conflicts is more important than getting all the rows

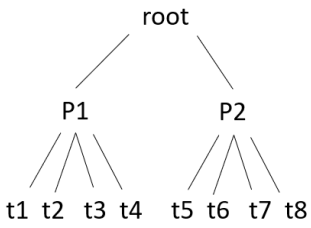
Hierarchical Locking

- Update locks prudentially extend the interval during which a resource is locked
- What to lock? An entire table? → reduces concurrency too much
- Locks can be specified with different granularities
  - e.g.: schema, table, fragment, page, tuple, field
- Objectives:
  - Locking the minimum amount of data
  - Recognizing conflicts as soon as possible
- Method: asking locks on hierarchical resouces by:
  - Requesting resources top-down until the right level is obtained
  - Releasing locks bottom-up

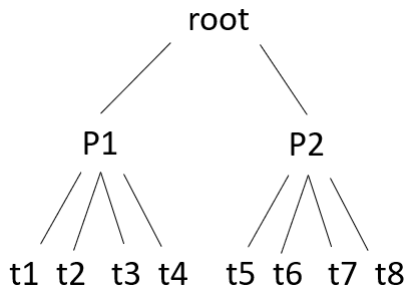


Coarser Granularity

Increased Concurrency

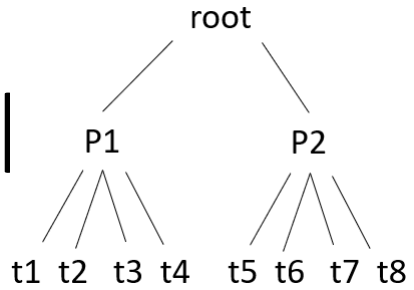


Intention Locking Scheme



- 5 Lock modes:
  - In addition to read (SHARED) locks (SL) and write (EXCLUSIVE) locks (XL)
- The new modes express the "intention" of locking at lower (finer) levels of granularity
  - **ISL**: Intention of locking a subelement of the current element in shared mode
  - **IXL**: Intention of locking a subelement of the current element in exclusive mode
  - **SIXL**: Lock of the element in shared mode with intention of locking a subelement in exclusive mode (SL+IXL)

Hierarchical Locking Protocol

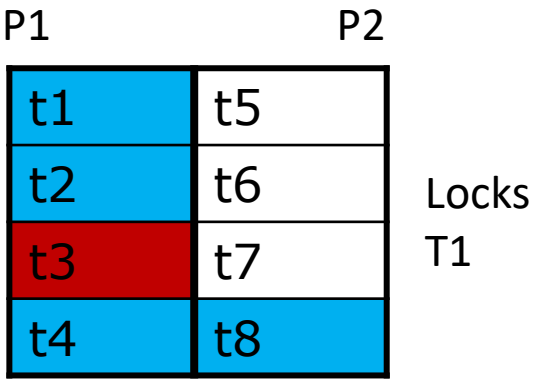


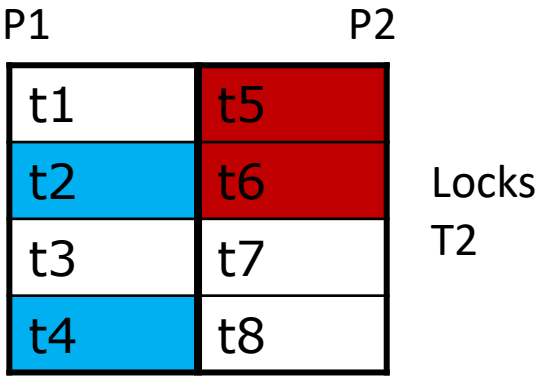
- Locks are requested starting from the root (e.g., starting from the whole table) and going down in the hierarchy
- Locks are released starting from the locked resource and going up in the hierarchy
- To request an SL or ISL lock on a non-root element, a transaction must hold an equally or more restrictive lock (ISL or IXL) on its “parent”
- To request an IXL, XL or SIXL lock on a non-root element, a transaction must hold an equally or more restrictive lock (SIXL or IXL) on its “parent”
- When a lock is requested on a resource, the lock manager decides based on the rules specified in the hierarchical lock granting table

Hierarchical lock granting table

Request	Resource state: free	ISL	IXL	SL	SIXL	XL
ISL	OK	OK	OK	OK	OK	No
IXL	OK	OK	OK	No	No	No
SL	OK	OK	No	OK	No	No
SIXL	OK	OK	No	No	No	No
XL	OK	No	No	No	No	No

Example





Root = TableX

Page 1 (P1): t1,t2,t3,t4 tuples

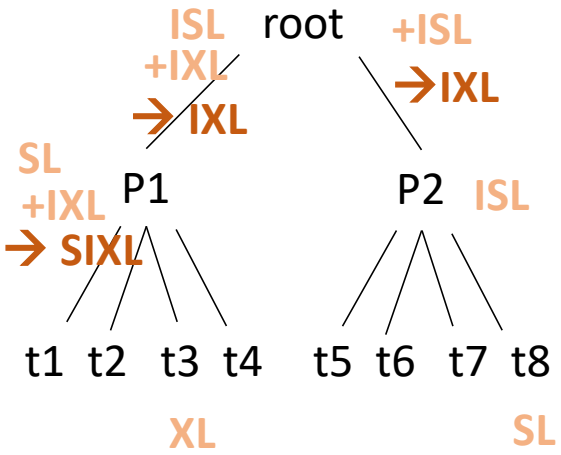
Page 2 (P2): t5,t6,t7,t8 tuples

Transaction 1: read(P1)
write(t3)
read(t8)

Transaction 2: read(t2)
read(t4)
write(t5)
write(t6)

They are NOT in r-w conflict!
(indipendently of the order)

Lock Sequences - Transaction 1



Transaction 1: read(P1)
write(t3)
read(t8)

Transaction 1:

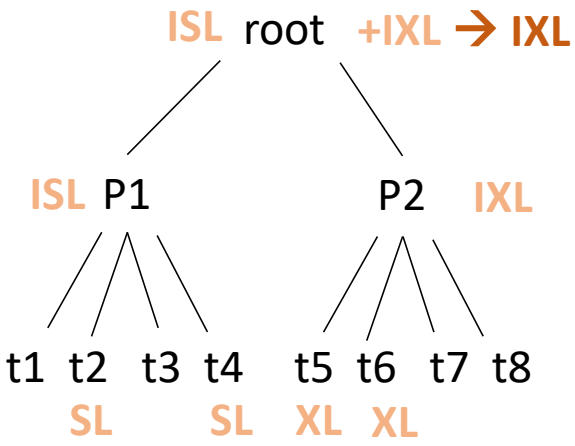
read(P1)

write(t3)

read(t8)

root	P1	...	t3	...	P2	...	t8
ISL_1	SL_1						
IXL_1	$SIXL_1$		XL_1				
IXL_1	$SIXL_1$		XL_1		ISL_1		SL_1

Lock Sequences – Transaction 2

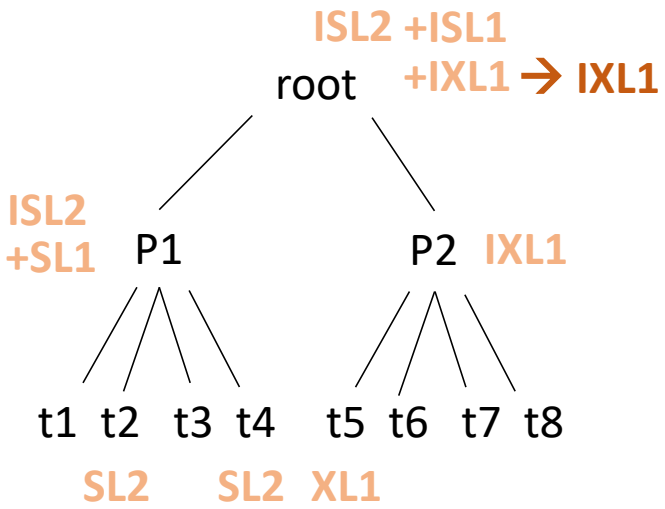


Transaction 2: read(t2)
read(t4)
write(t5)
write(t6)

Transaction 2:

	root	P1	..	t2	..	t4	P2	..	t5	t6
read(t2)	ISL_2	ISL_2		SL_2						
read(t4)	ISL_2	ISL_2		SL_2		SL_2				
write(t5)	IXL_2	ISL_2		SL_2		SL_2	IXL_2		XL_2	
write(t6)	IXL_2	ISL_2		SL_2		SL_2	IXL_2		XL_2	XL_2

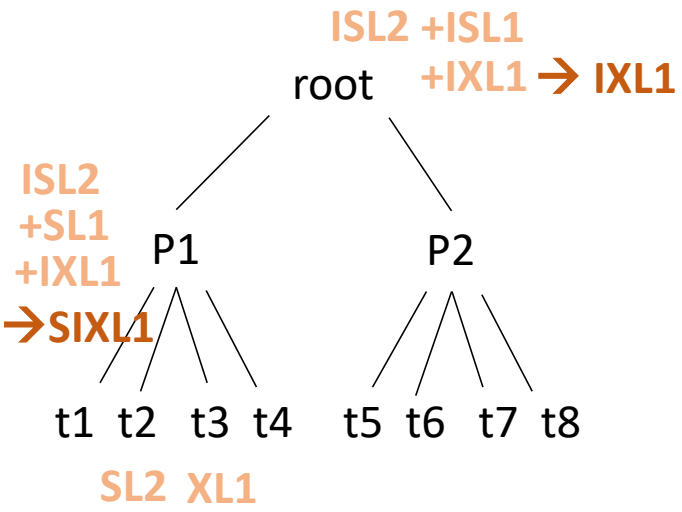
Lock Sequences – interleaved execution



Request	Resource state: free	ISL	IXL	SL	SIXL	XL
ISL	OK	OK	OK	OK	OK	No
IXL	OK	OK	OK	No	No	No
SL	OK	OK	No	OK	No	No
SIXL	OK	OK	No	No	No	No
XL	OK	No	No	No	No	No

	root	P1	..	t2	..	t4	P2	..	t5	t6
$read_2(t2)$	ISL_2	ISL_2		SL_2						
$read_1(P1)$	$ISL_{1,2}$	ISL_2, SL_1		SL_2						
$write_1(t5)$	ISL_2, IXL_1	ISL_2, SL_1		SL_2			IXL_1		XL_1	
$read_2(t4)$	ISL_2, IXL_1	ISL_2, SL_1		SL_2		SL_2	IXL_1		XL_1	
...										

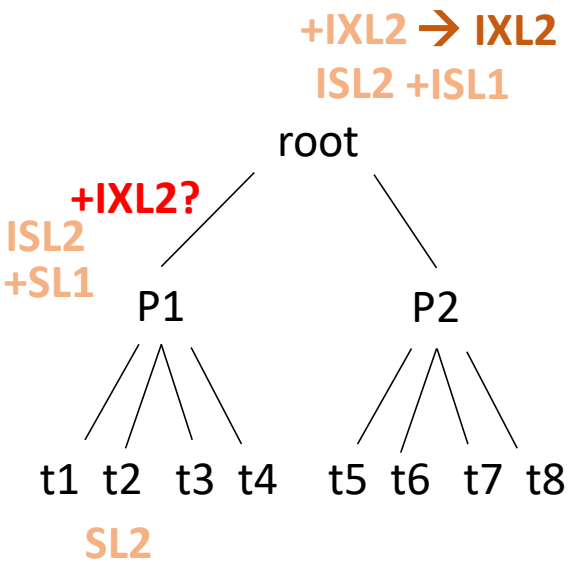
Lock Sequences – let's change t5 into t3



Request	free	ISL	IXL	SL	SIXL	XL
	Resource state					
ISL	OK	OK	OK	OK	OK	No
IXL	OK	OK	OK	No	No	No
SL	OK	OK	No	OK	No	No
SIXL	OK	OK	No	No	No	No
XL	OK	No	No	No	No	No

	root	P1	..	t2	..	t4	P2	..	t5	t6
$read_2(t2)$	ISL_2	ISL_2		SL_2						
$read_1(P1)$	$ISL_{1,2}$	ISL_2, SL_1		SL_2						
$write_1(\mathbf{t3})$	ISL_2, IXL_1	ISL_2, $SIXL_1$		SL_2		XL_1				

Lock Sequences – let's change w1 into w2



Request	Resource state: free	ISL	IXL	SL	SIXL	XL
ISL	OK	OK	OK	OK	OK	No
IXL	OK	OK	OK	No	No	No
SL	OK	OK	No	OK	No	No
SIXL	OK	OK	No	No	No	No
XL	OK	No	No	No	No	No

	root	P1	..	t2	..	t4	P2	..	t5	t6
$read_2(t2)$	ISL_2	ISL_2		SL_2						
$read_1(P1)$	$ISL_{1,2}$	ISL_2, SL_1		SL_2						
$write_2(\mathbf{t3})$	ISL_1, IXL_2	ISL_2, SL_1 (←IXL2)		SL_2						

Conflict!
T2 waits!

Concurrency Control Based on Timestamps

- Locking is also named pessimistic concurrency control because it assumes that collisions (transactions reading-writing the same object concurrently) will arise
  - Assumption: conflicts occur → lock the records to prevent them
- Alternative and complementary to 2PL (and to locking in general) are optimistic concurrency control methods
  - Assumption: conflicts are rare → run the transaction and validate the operations before commit (normal validation) or before each operation (early validation)
- Timestamp:
  - Identifier that defines a total ordering of the events of a system
- Each transaction has a timestamp representing the time at which the transaction begins so that transactions can be ordered by "birth date": smaller index → older transaction
- A schedule is accepted only if it reflects the serial ordering of the transactions induced by their timestamps

TS concurrency control principles

- The scheduler has two counters: RTM(x) and WTM(x) for each object
  - RTM (x) = timestamp of the transaction with the highest ts that has read x
  - WTM (x) = timestamp of the transaction that did the last write on x

e.g., after r2(x), **r3(x)**, r1(x) RTM=3

- The scheduler receives read/write requests tagged with the timestamp of the requesting transaction:
  - $r_{ts}(x)$:
    - If ts < WTM(x) the request is rejected and the transaction is **killed**
    - Else, access is granted and RTM(x) is set to max(RTM(x), ts)
  - $w_{ts}(x)$:
    - If ts < RTM(x) or ts < WTM(x) the request is rejected and the transaction is **killed**
    - Else, access is granted and WTM(x) is set to ts

e.g., w1(x), w3(x), **r2(x)**

e.g., w2(x), **w1(x)**
or r2(x), **w1(x)**

- Many transactions are killed

Example

Assume RTM(x) = 7 → previously: $r_7(x)$

WTM(x) = 4 → previously: $w_4(x)$

Request	Response	RTM(x)	WTM(x)	
		7	4	
$r_6(x)$	ok (6>4)			
$r_8(x)$	ok (8>4)	8		
$r_9(x)$	ok (9>4)	9		
$w_8(x)$	no (8<9)			T8 killed
$w_{11}(x)$	ok (11>9 and 11>4)		11	
$r_{10}(x)$	no (10<11)			T10 killed

2PL vs. TS

They are incomparable

- Schedule in TS but not in 2PL
  - r1(x) w1(x) r2(x) w2(x) r0(y) w1(y)



↔ r0(y) r1(x) w1(x) w1(y) r2(x) w2(x)

- Schedule in 2PL but not in TS
  - r2(x) w2(x) r1(x) w1(x)
  - This is serial!
- Schedule both in TS and in 2PL (and not serial)
  - r1(x) r2(y) w2(y) w1(x) r2(x) w2(x)



↔ r1(x) w1(x) r2(y) w2(y) r2(x) w2(x)

TS and CSR

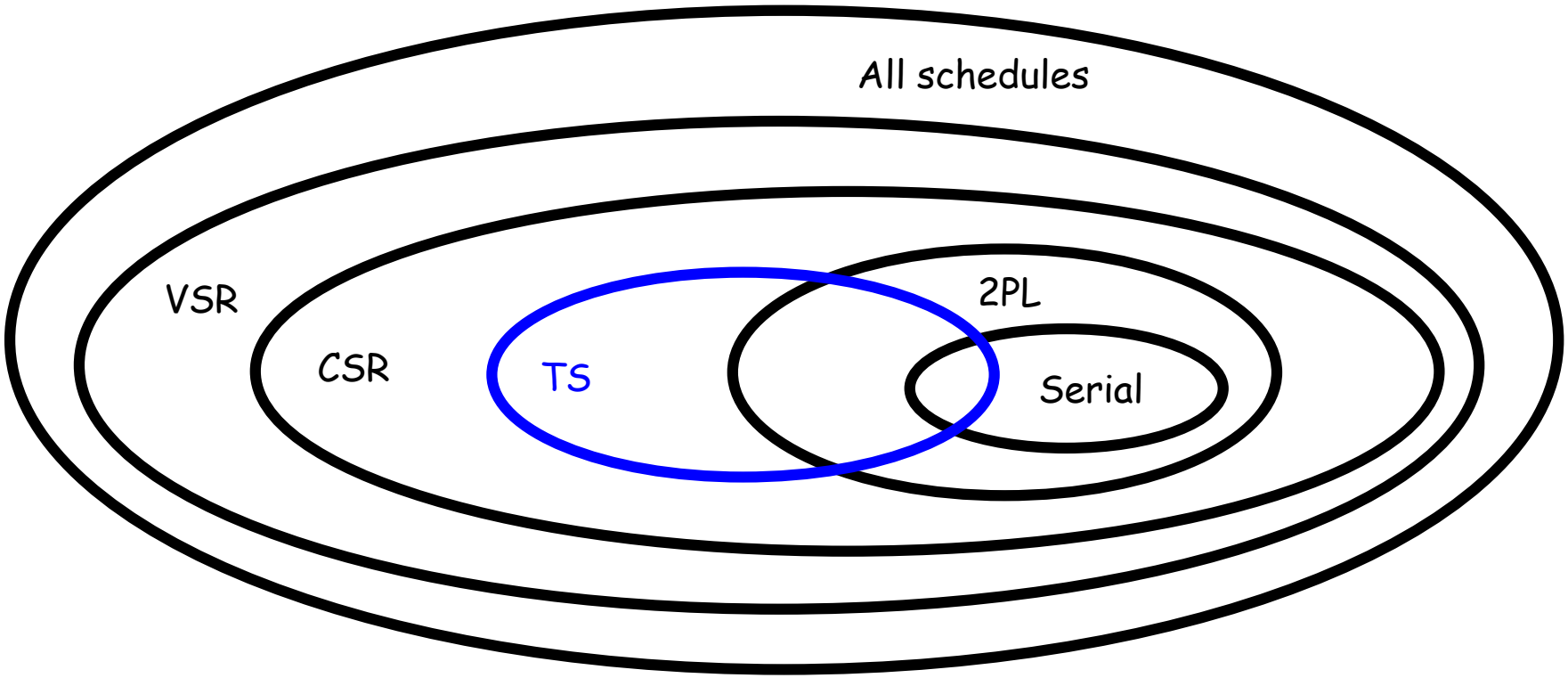
- **TS => CSR**
  - Let S be a TS schedule of T1 and T2
  - Suppose S is not CSR, which implies that it contains a cycle between T1 and T2

    r_1-w_2 w_1-w_2 w_1-r_2

    - S contains $op_1(x)$, $op_2(x)$ where at least one of the op_i is a write
    - S contains also $op_2(y)$, $op_1(y)$ where at least one of the op_i is a write

    r_2-$\mathbf{w_1}$ w_2-$\mathbf{w_1}$ w_2-$\mathbf{r_1}$
  - When $op_1(y)$ arrives:
    - If $op_1(y)$ is a read, T1 is killed by TS because it tries to read a value written by a younger transaction [ts < WTM(y)] [1 < 2] → CONTRADICTION
    - If $op_1(y)$ is a write, T1 is killed no matter what $op_2(y)$ is because it tries to write a value already read or written by a younger transaction [ts < R/WTM(y)] [1 < 2] → CONTRADICTION

CSR, VSR, 2PL and TS



TS and dirty reads

- Basic TS-based control considers only committed transactions in the schedule, aborted transactions are not considered (**commit-projection hypothesis**)
- If **aborts** occur, dirty reads may happen
- To cope with dirty reads, a **variant** of basic TS must be used
  - A transaction T_i that issues a $\mathbf{r_{ts}(x)}$ **such that *ts* > WTM(x)** (i.e., acceptable) has its read operation **delayed** until the transaction T' that wrote the values of x has committed or aborted
    - Similar to long duration write locks
    - But…buffering operations introduces delays

e.g., w1(x), r2(x) T1 aborts!

2PL vs. TS

- The serialization order with 2PL is imposed by **conflicts**, while in TS it is imposed by the **timestamps**
- In 2PL transactions can be actively **waiting**. In TS they are **killed** and restarted
- The necessity of waiting for commit of transactions causes long delays in strict 2PL
- 2PL can cause deadlocks, TS can be used to prevent deadlocks with the wound-wait and wait-die schemes
- Restarting a transaction costs more than waiting: 2PL wins!
- Commercial systems implement a mix of optimistic and pessimistic concurrency control (e.g., Strict 2PL or 2PL + Multi Version TS)

Reducing kill rate: Thomas Rule

- The scheduler has two counters: RTM(x) and WTM(x) for each object
- The scheduler receives read/write requests tagged with timestamps:
  - $r_{ts}(x)$:
    - If ts < WTM(x) the request is rejected and the transaction is killed
    - Else, access is granted and RTM(x) is set to max(RTM(x), ts)
  - $w_{ts}(x)$:
    - If ts < RTM(x) the request is rejected and the transaction is killed
    - Else, **if ts < WTM(x)** then our write is "obsolete": it can be **skipped**
    - Else, access is granted and WTM(x) is set to ts

ts < RTM(x)

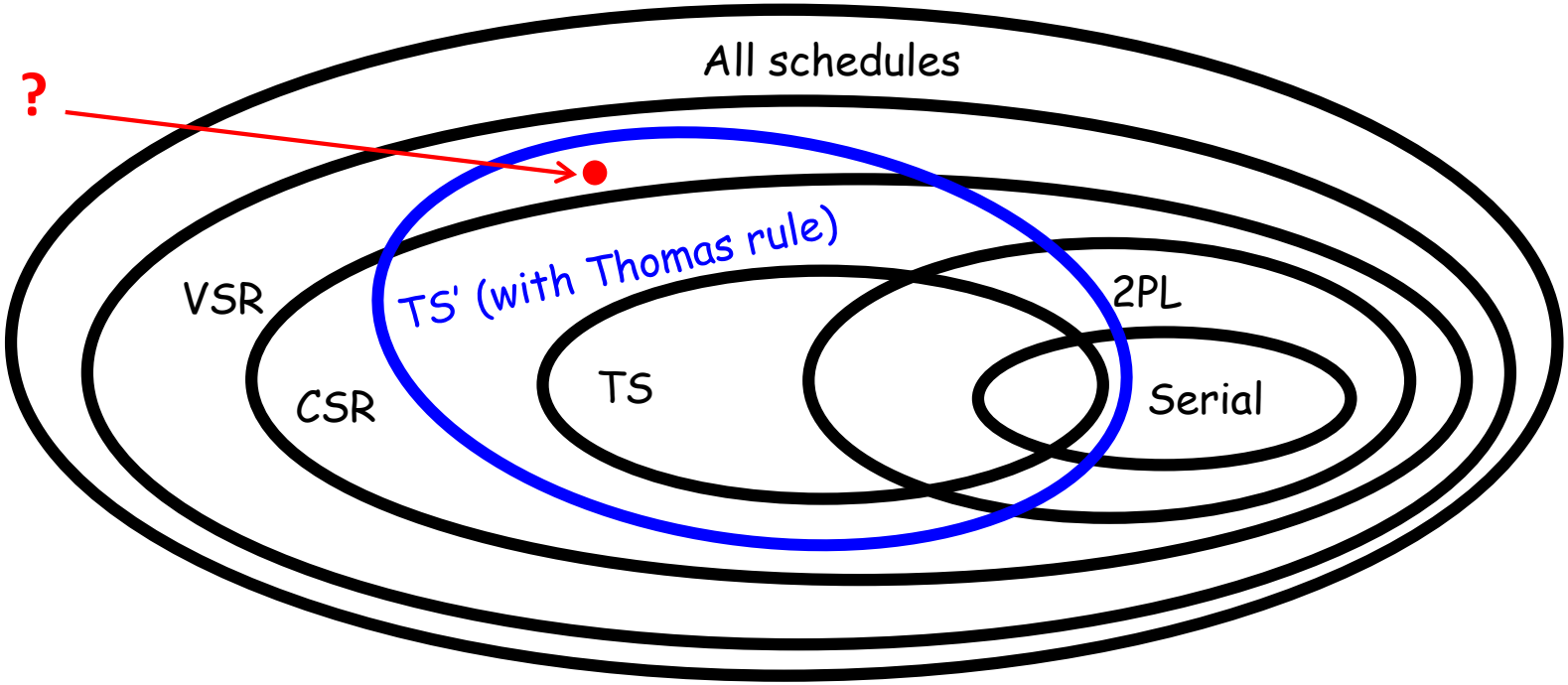
e.g., r2(x), w1(x)

ts < WTM(x)

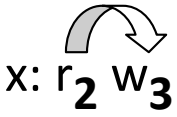
e.g., w2(x), **w1(x)**

- Rationale: skipping a write on an object that has already been written by a younger transaction, without killing the transaction
- Does this modification affect the taxonomy of the serialization classes?

TS' (TS with Thomas Rule)



- $r_1(y)\ r_2(x)\ w_3(y)\ w_2(y)\ w_3(x)\ w_4(y)$



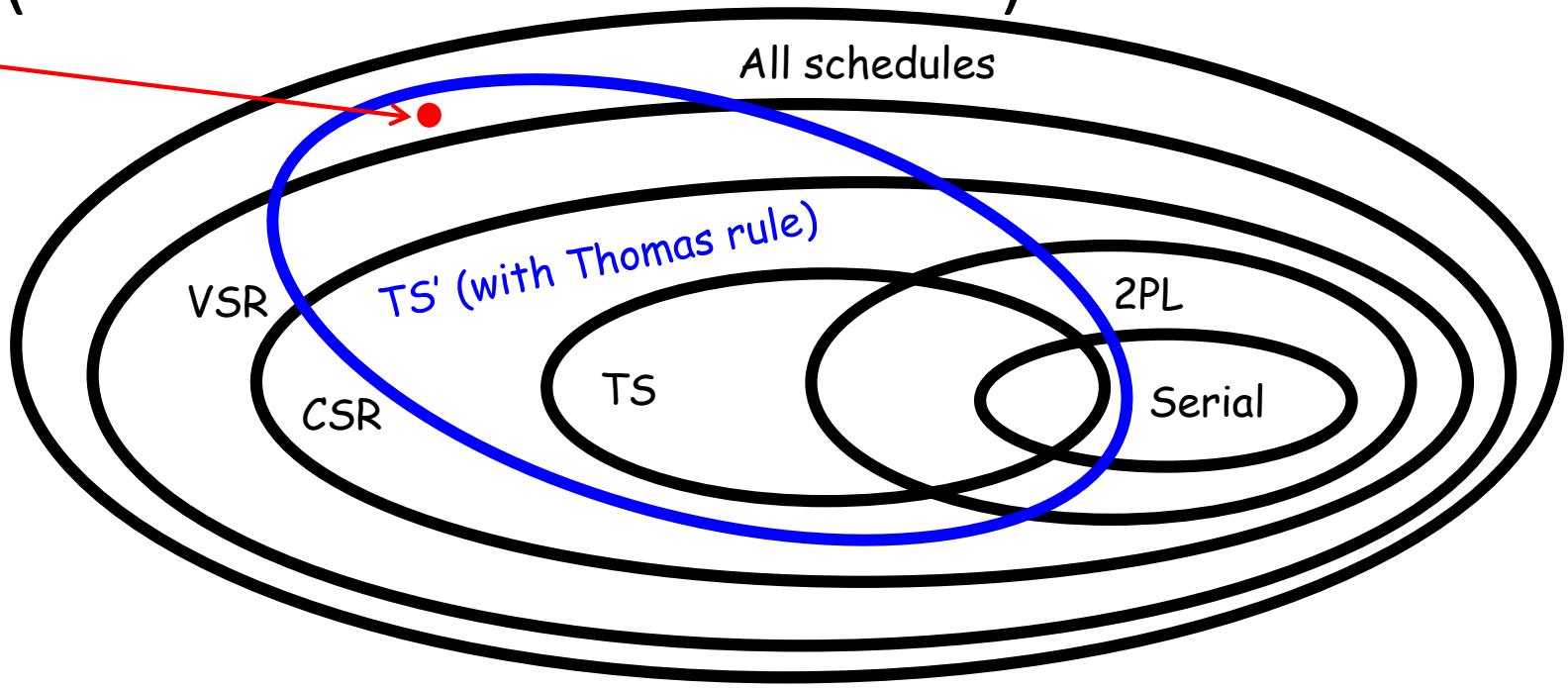
Not CSR!



$w_2(y)$ is an obsolete write and thus skipped

It reflects the behavior of: $r_1(y)\ r_2(x)\ w_2(y)\ w_3(y)\ w_3(x)\ w_4(y)$

TS' (TS with Thomas Rule)



- $w_2(x)\ w_1(x)\ r_2(x)$
  - $w_1(x)$ skipped by Thomas Rule
- The schedule is not in VSR
  - T1, T2 different final write relation
  - T2, T1 different reads-from relation for $r_2(x)$

$w_1(x)\ w_2(x)\ r_2(x)$

$w_2(x)\ r_2(x)\ w_1(x)$

Multiversion Concurrency Control

- Idea: writes generate new versions, reads access the “right” version
- Writes generate new copies, each one with a new WTM. Each object x always has N>=1 active versions
  - The i-th version of x has its write timestamp $WTM_i(x)$
- There is a unique global RTM(x)
- Old versions are discarded when there are no transactions that need their values

Example **in theory**: TS-Multi allowing **unordered writes**

- Mechanism:
  - **$r_{ts}(x)$ is always accepted.** A copy x_k is selected for reading such that:
    - If *ts* >= $WTM_N(x)$, then k = N
    - Else take k such that $WTM_k(x) <= ts < WTM_{k+1}(x)$
  - $w_{ts}(x)$:
    - If *ts* < RTM(x) the request is rejected
    - Else a new version is created for timestamp *ts* (N is incremented)
      - $WTM_1(x)$, …, $WTM_N(x)$ are the new versions, kept sorted from oldest to youngest
      - NB: this version shows what can be done in theory but is not the one used in the exercises

Example **in theory**: TS-Multi allowing **unordered writes**

Assume RTM(x) = 7, N=1, $WTM_1(x)$ = 4

Request	*Response*	*RTM(x)*	*WTM(x)*
$r_6(x)$	ok	7	$WTM_1(x)$ = 4, N=1
$r_8(x)$	ok	8	
$r_9(x)$	ok	9	
$w_8(x)$	no - T_8 killed		
$w_{11}(x)$	ok		$WTM_2(x)$ = 11, N=2
$r_{10}(x)$	ok on x_1 (not killed)	10	
$r_{12}(x)$	ok on x_2	12	
$w_{14}(x)$	ok		$WTM_3(x)$ = 14, N=3
$w_{13}(x)$	ok (not killed)		$WTM_3(x)$=13, N=3
			$WTM_4(x)$=14, N=4

requires resorting

2nd version (used in practice): TS-Multi **under Snapshot Isolation**

- Mechanism:
  - $r_{ts}(x)$ is always accepted. A copy x_k is selected for reading such that:
    - If *ts* >= $WTM_N(x)$, then k = N
    - Else take k such that $WTM_k(x) <= ts < WTM_{k+1}(x)$
  - $w_{ts}(x)$:
    - If *ts* < RTM(x) **or *ts* < $WTM_N(x)$** the request is rejected
    - Else a new version is created for timestamp *ts* (N is incremented)
      - $WTM_1(x)$, …, $WTM_N(x)$ are the new versions, kept sorted from oldest to youngest
    - NB: this version is used in real systems based, e.g., on *snapshot isolation* (see later) and in the exercises

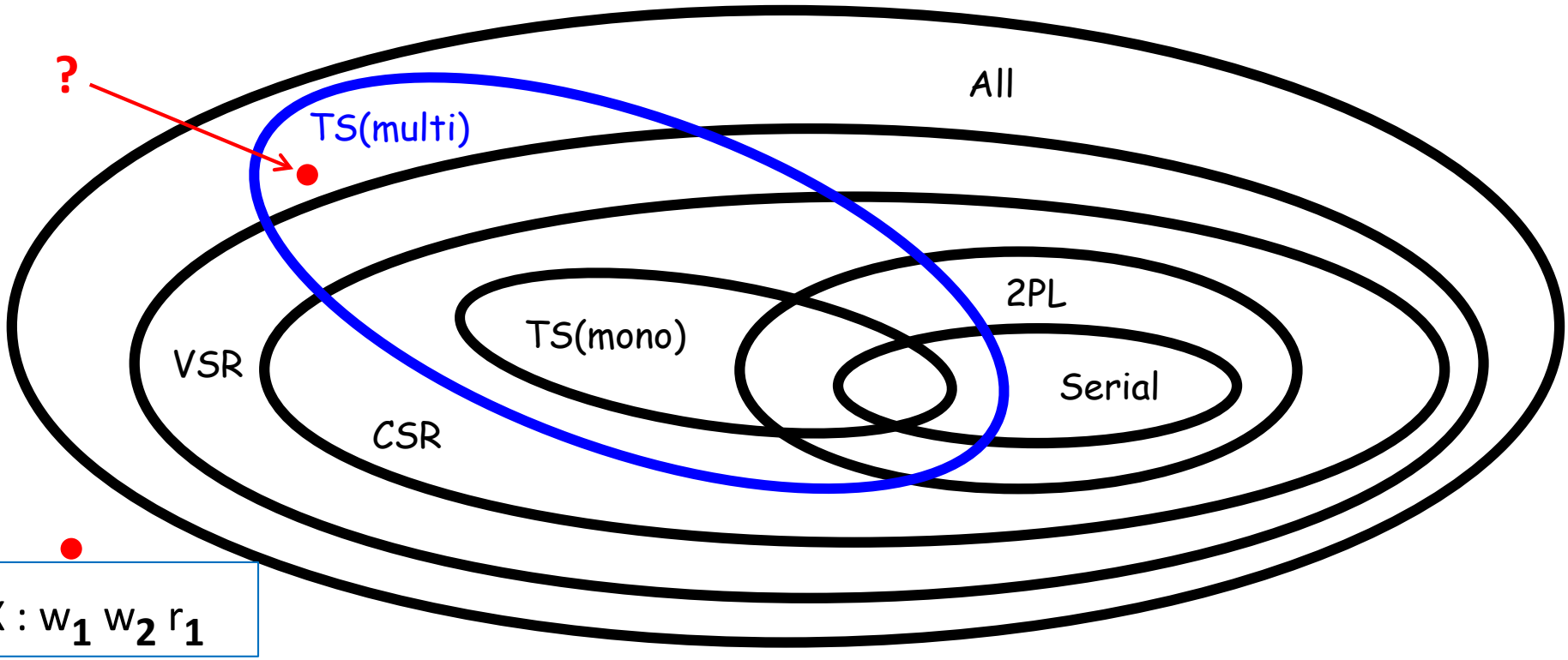
Example in practice (**for the exam**): TS-Multi **under Snapshot Isolation**

Assume RTM(x) = 7, N=1, $WTM_1(x) = 4$

Request	*Response*	*RTM(X)*	*WTM(X)*
$r_6(x)$	ok	7	$WTM_1(x) = 4$, N=1
$r_8(x)$	ok	8	
$r_9(x)$	ok	9	
$w_8(x)$	no - T_8 killed		
$w_{11}(x)$	ok		$WTM_2(x) = 11$, N=2
$r_{10}(x)$	ok on x_1	10	
$r_{12}(x)$	ok on x_2	12	
$w_{14}(x)$	ok		$WTM_3(x) = 14$, N=3
$w_{13}(x)$	no - T_{13} killed*		

* in the exercises!!!

CSR, VSR, 2PL, TSmono, TSmulti



X : w_1 w_2 r_1

- Versions: X0 (original value), X1, X2
- r_1 reads X1 [$WTM_k(x) <= ts < WTM_{k+1}(x)$]

- The schedule is not in VSR
  - T1, T2 different reads-from relation for $r_1(x)$
  - T2, T1 different final write relation

w_1 r_1 w_2

w_2 w_1 r_1

Snapshot Isolation (SI)

- The realization of multi-TS gives the opportunity to introduce into DBMSs (e.g., Oracle, MySQL, PostgreSQL, MongoDB, Microsoft SQL Server) another isolation level, SNAPSHOT ISOLATION
- In this level, no RTM is used on the objects, only WTMs
- Every transaction **reads the version consistent with its timestamp** (i.e., the version that existed when the transaction started a.k.a. **snapshot**), and **defers writes to the end**
- **Write**: when a transaction attempts to write e.g., on a row, it first checks whether any other transactions have modified that row since it began. If there has been a modification (i.e., if the snapshot view is no longer valid), the transaction is rolled back or retried, depending on the DBMS's implementation.
- Read operations in a transaction do not block write operations in other transactions. Transactions can read data without waiting for other transactions to complete, improving performance and concurrency.
- It is yet another case of optimistic concurrency control

Anomalies in Snapshot Isolation



- Snapshot isolation **does not guarantee serializability**
  - T1: update Balls set Color=White where Color=Black
  - T2: update Balls set Color=Black where Color=White
- Serializable executions of T1 and T2 will produce a final configuration with balls that are either all white or all black
- An execution under Snapshot Isolation in which the two transactions start with the same snapshot will just swap the two colors

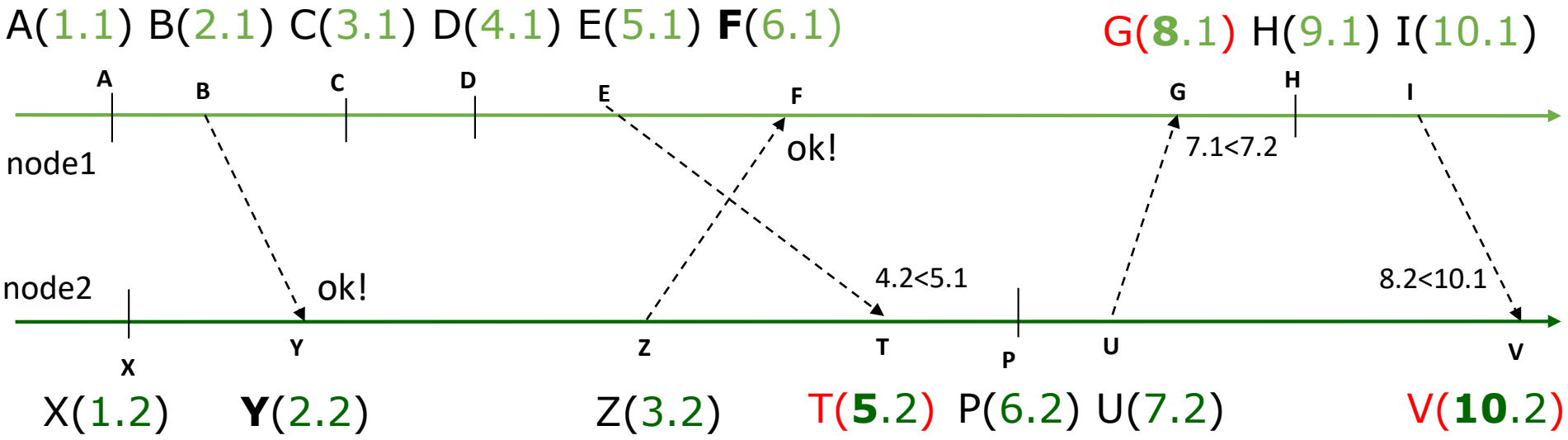


- This **anomaly** is called **write skew**

Assigning timestamps in distributed systems

- Timestamp: an indicator of the “current time”
- Assumption: no “global time” is available
- Mechanism: a system’s function gives out timestamps on requests
- Syntax: timestamp = **event-id.node-id**
  - event-ids are unique at each node
- Note that the notion of time is “lexical”:
  timestamp 5.1 “occurs before” timestamp 5.2
- Synchronization: send-receive of messages
  - for a given message m, send(m) precedes receive(m)
- **Algorithm (Lamport method): cannot receive a message from “the future”**, if this happens the “bumping rule” is used to bump the timestamp of the receive event beyond the timestamp of the send event
- **Mnemonically**: if I receive a message from you that has a timestamp greater than my last emitted one I update my current timestamp to exceed yours

Example of timestamp assignment



- Events Y and F represent messages received "from the present or past" → OK the local timestamp is aligned without bumping
- Events T, G and V represent messages received "from the future" → The local timestamp is incremented (bumped) accordingly leaving a "hole" in the local event sequence
- Note that the timestamp of the receive events T, G and V is generated so as to exceed that of the send event: G(8.1)>U(7.2)

Concurrency control in some commercial DBMS

- ORACLE
- https://docs.oracle.com/cd/B19306_01/server.102/b14220/consist.htm

- MYSQL
- https://dev.mysql.com/doc/refman/8.0/en/locking-issues.html

- IBM DB2
- https://www.ibm.com/support/knowledgecenter/en/SSEPGG_9.7.0/com.ibm.db2.luw.admin.perf.doc/doc/c0054923.html

- MONGODB
- https://docs.mongodb.com/manual/faq/concurrency/